

Pithon

Computer Controlled IO
Based on Raspberry Pi

Fang Yuan
(yfang@nju.edu.cn)

July 2026 in Nanjing University

Content

1	The Linux Operating System	2
1.1	Overview of Operating Systems	2
1.2	Introduction to Linux	3
1.3	Using Linux	8
1.3.1	The Command-Line Format	8
1.3.2	Users and Permissions	9
1.3.3	Directory Operations	11
1.3.4	File Operations	12
1.3.5	Process Control	16
1.3.6	Modifying Process States	17
1.3.7	Foreground and Background Execution	17
1.3.8	I/O Redirection and Pipelines	19
1.4	Text Editors	20
1.5	Network Connectivity	22
1.5.1	Remote Terminal Access	22
1.5.2	Remote Desktop Access	24
1.5.3	Remote Network Storage (NFS)	26
1.6	Chapter Summary	26
2	Very Short Python Introduction	28
2.1	Introduction	28
2.2	Python Environment	29
2.3	Python Basics	30
2.3.1	Running Your First Python Program	30
2.3.2	Interactive Mode	31
2.3.3	Numerical Variables	32
2.3.4	Strings	34
2.3.5	Boolean Logic	36

2.3.6	Functions	37
2.3.7	Order of Operations	38
2.4	Data Structures	41
2.4.1	Lists	41
2.4.2	Tuples	43
2.4.3	Sets	43
2.4.4	Dictionaries	44
2.5	Python Program Architecture	44
2.5.1	Program Structure	44
2.5.2	Conditional Execution	45
2.5.3	Loops and Iterations	47
2.5.4	Functions and Scope	49
2.5.5	Modules and Package Management	50
2.6	Summary	51
3	I/O Interfaces on Raspberry Pi	53
3.1	Raspberry Pi	53
3.1.1	Short Introduction	53
3.2	Low-Level Hardware Expansion Interfacing	54
3.3	I/O System Usages	56
3.4	GPIO Modules for Python	57
3.4.1	Install a GPIO Module	57
3.4.2	Pin Numbering	58
3.4.3	Set up Pins	59
3.4.4	Use GPIO as Input	60
3.4.5	Untrasonic Ranging	63
3.5	Using PWM	64
3.5.1	What is a PWM	64
3.5.2	Accessing PWM in Userspace	64
3.5.3	PWM Controlling using RPi.GPIO	65
3.6	I2C Module	66
3.6.1	Single Segments Display	66
3.6.2	Multiple Display	66
3.7	Addressable LED Strip WS2812	71

List of Figures

1.1	Selected Linux Distributions	6
1.2	OS Market Share Worldwide	7
1.3	VIM Mode Transitions	20
1.4	XFCE4 Graphical Desktop Environment Session on a Raspberry Pi . . .	25
3.1	Different Models of Raspberry Pi	53
3.2	Alternative Single-Board Computer Configurations	54
3.3	Connecting an LED and a Switch to GPIOs	57
3.4	Ultrasonic Ranging	63
3.5	7-Segment LED Display	66
3.6	Generalized I2C Connection Diagram	67
3.7	Standard I2C Bus Signaling and Timing Constraints	67
3.8	WS2812 Addressable LED Strip	72

List of Tables

1.1	Linux File Type Identifiers	10
1.2	Common File Management Wildcards	13
1.3	Common ‘tar‘ Command Options	15
1.4	Standard Linux System Signals	18
1.5	Standard Shell I/O Redirection Operators	19
1.6	Summary of Core Linux Commands	27
2.1	String Format Specifiers in Python	34
2.2	Operator Precedence Hierarchy in Python Expressions	39
2.3	Bitwise Operators and Expressions	40
2.4	Compound Assignment Statements	41
2.5	Reserved Keywords in Python Programming	51
2.6	Globally Available Built-in Interpreter Functions	52
3.1	Physical GPIO Pin Mapping of the Raspberry Pi	55
3.2	Supported GPIO Pins on Raspberrypi	72

Chapter 1

The Linux Operating System

1.1 Overview of Operating Systems

An operating system (OS) is the core software that manages computer hardware and software resources. Positioned between the computer hardware and the user, it serves as a bridge for communication between the two. Users input commands through the OS user interface, and the operating system interprets these commands to drive hardware devices, thereby fulfilling user requests.

The most common operating systems we use on personal computers are Windows and Linux. Apple personal computers (the Mac series) run macOS, which is a branch of the UNIX operating system family. Smartphones and tablets also require operating systems. Except for Apple's iPhones and iPads, which run iOS (another UNIX derivative), almost all other smartphones and tablets currently on the market run Android. Crucially, the underlying kernel of the Android system is Linux.

A modern operating system typically consists of several subsystems: process management, memory management, file systems, device drivers, and network subsystems. Because a general-purpose operating system lacks a specific, single application requirement, it must support a broader range of hardware and software to accommodate diverse use cases. It allocates system resources dynamically based on the overall user experience. Consequently, general-purpose computer operating systems are usually **time-sharing operating systems**, which prioritize fairness and rationality in resource allocation over strict timing constraints. In contrast, **embedded operating systems** are typically bound by stringent real-time requirements.

1.2 Introduction to Linux

Linux is an operating system that adheres to the GNU General Public License (GPL). In April 1991, Linus Torvalds, then a graduate student at the University of Helsinki in Finland, began writing this operating system out of personal interest while studying an operating system course, subsequently releasing it to the Internet.



Linus Torvalds

Once published, the system sparked an immediate and enthusiastic response from programmers worldwide. Since then, its development has progressed unstoppably, turning Linux into a highly influential operating system today. Thousands of programmers have contributed code to Linux. The list of Linux kernel maintainers alone spans over four thousand lines, to say nothing of the massive community of application developers.

The primary characteristics and advantages of the Linux system can be summarized as follows:

1. Open Source Code

The Linux kernel follows an open-source license, and the vast majority of software bundles used across various distributions are open-source. Open-source software not only facilitates convenient development but, more importantly, provides robust security guarantees. Under open inspection, it is exceptionally difficult for malicious code to hide. Software engineers and Linux users worldwide actively contribute suggestions and patches to the open-source community, ensuring that bugs are resolved rapidly once discovered. As a result, the stable and reliable Linux operating system has become the premier choice for the vast majority of enterprise servers.

2. Reliability and Stability

Linux operates in protected mode, enforcing a strict separation between the kernel space and user space memory addresses. It implements rigorous read and write permission controls for different users, protected subsystems, and core authorization structures. These comprehensive security measures, paired with good administrative habits, allow a Linux system to run continuously and stably for long periods without rebooting.

3. Excellent Portability

Linux software development strictly adheres to the **POSIX** (Portable Operating System Interface) standard, meaning applications can be migrated across different platforms with little to no modification. POSIX is a collective term for a series of interrelated standards established by the IEEE and ISO/IEC. During the mid-1980s, various competing distributions emerged throughout the evolution of UNIX. To maintain compatibility across computer systems, POSIX defined standard application programming interfaces (APIs), command-line shells, and utility interfaces based on UNIX historical practices. This ensures portability at the source-code level. Today, the Linux kernel supports dozens of processor architectures (including Intel x86, ARM, and PowerPC) and hundreds of hardware platforms, making it highly portable to almost all mainstream processor environments.

4. Device Independence

Linux famously adheres to the philosophy that **“everything is a file.”** All physical hardware devices are abstracted and treated uniformly as files. The kernel provides a standardized interface call system for each device. Consequently, applications can manipulate and utilize these hardware components exactly as they would open, read, or write to a regular file, without needing to understand the underlying hardware complexities.

The core of device independence lies in the adaptability of the kernel. This provides a major advantage: user programs remain entirely decoupled from physical hardware configurations. When peripheral hardware is upgraded or swapped, the application code requires no modification, significantly enhancing peripheral allocation flexibility and hardware resource efficiency.

5. Multiple User Interfaces

From its early days as a minimalist text-only interface, Linux has evolved to offer both rich graphical user interfaces (GUIs) and powerful command-line interfaces (CLIs) side by side, allowing it to adapt to varying system resource constraints. The traditional command-line interface leverages the robust scripting capabilities of the shell, making it fully capable of handling server workloads even on computers completely lacking graphical hardware. Meanwhile, desktop Linux environments offer numerous GUI options (such as GNOME or XFCE), allowing users to fully customize their personal workspaces.

Note that the command-line interface is a powerful choice, not an inherent

limitation of Linux. For experienced users, it provides a far more efficient workflow. For consumer use cases, modern Linux desktop graphical environments are fully capable of meeting regular desktop computing needs.

6. Multi-User and Multi-Tasking Support

UNIX was originally designed from the ground up to accommodate multiple concurrent users on a single computer system. Linux inherited this architecture natively. Beyond allocating isolated system resources for individual accounts like standard personal computer operating systems, Linux can provide distinct, simultaneous terminal environments for different users. This allows multiple people to work on the exact same physical computer concurrently.

This behavior differs fundamentally from operating systems designed strictly for personal single-user workstations. While network advancements allow multiple users to log into a single server over a network today, doing so practically means an individual is utilizing his or her own computer, which differs conceptually from the native multi-user terminal design of Linux.

Multi-tasking is a hallmark of modern operating systems, referring to a computer's ability to execute multiple programs concurrently in an mutually isolated manner. The Linux scheduler independently allocates CPU resources and memory spaces to each task, ensuring they do not interfere with one another. The failure or crash of a single task generally does not impact other running processes, providing a fundamental guarantee for overall system reliability.

7. Comprehensive Networking Capabilities

As a direct descendant of the UNIX architectural lineage, Linux possesses all the technological network advantages of UNIX systems. UNIX was the first operating system to implement the TCP/IP suite natively. Linux completely inherited this network protocol stack, providing full TCP/IP client and server capabilities alongside powerful network communication facilities. Deploying standard network services on top of a Linux OS is exceptionally straightforward.

8. Extensive File System Support

Linux utilizes a **Virtual File System** (VFS) layer to abstract and support widely differing file systems. It can natively recognize almost all known disk partition formats, allowing software to read and write data across different partitions seamlessly without modifications to the application code.

9. Convenient Development and Maintenance Facilities

Linux distributions bundle a massive array of programming language development tools and rely heavily on scripting languages. This script-centric design not only streamlines software development but forms the bedrock of system maintainability. Most Linux background services are managed via scripts, and server parameters are configured through plain-text configuration files. Even the core system initialization and boot flows are orchestrated by scripts. This enables system administrators to manage the entire machine using any text editor they prefer.

Today, the Linux operating system spans every scale of computing hardware—from tiny wearable items and card-sized IoT devices up to massive supercomputers and distributed enterprise clusters. Outside of the mainstream personal desktop market, Linux maintains a dominant monopoly in almost every computing domain it occupies.

The Linux community is incredibly active. Historically, a new kernel version was released roughly every six months; recently, development cycles have become even faster. Kernel version 7.1.3 was released on July 4th 2026. A Long-Term Support (LTS) 6.18.38 is updated also on July 4th 2026. Standard kernels generally receive a two-year maintenance window, but the LTS kernel (6.18) will be officially maintained through December 2028.



Figure 1.1: Selected Linux Distributions

Linux is categorized as **free software**, emphasizing **freedom** (as in “free speech”) rather than price (as in “free beer”). Because the English word “free” carries both meanings, it is frequently misinterpreted as merely zero-cost software. To distinguish the two concepts, pure zero-cost proprietary software is referred to as **freeware**. Due to the terms of the GPL license, nearly all software components within a Linux distribution

provide open source-code access free of charge. Many open-source licenses explicitly permit users to modify and redistribute the code. Consequently, provided the license terms are met, anyone can compile and assemble these software packages into their own custom operating system, which has given rise to a vast ecosystem of diverse Linux distributions (Figure 1.1).

Since its inception, Linux has rapidly captured the global server market due to its structural stability. Today, 100% of the top 500 fastest supercomputers in the world run Linux.¹

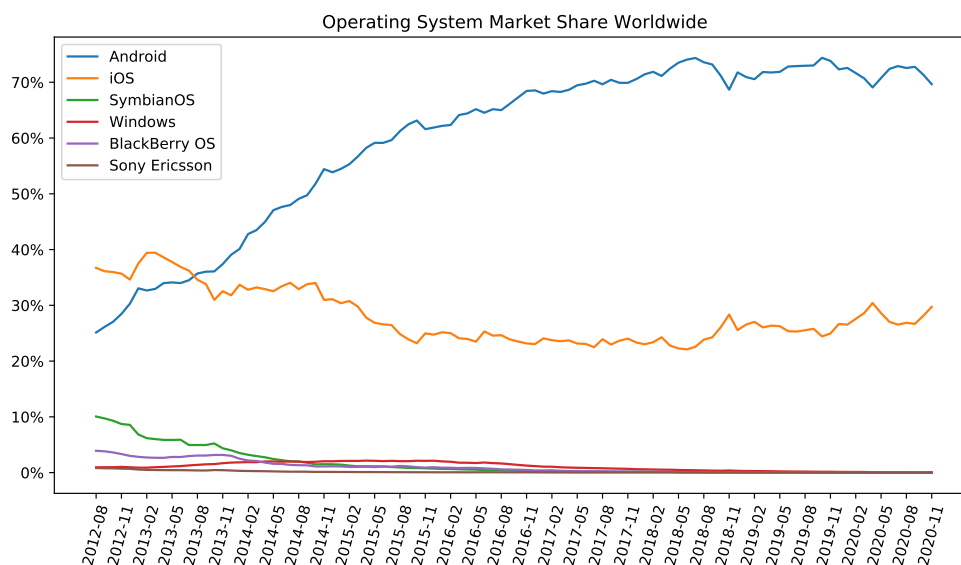


Figure 1.2: Operating System Market Share on Smartphones and Tablets²

Linux’s greatest success in the consumer market is perhaps the mobile device market, with Android being one of the most dominant operating systems on smartphones and very popular on tablets (Figure 1.2) and, more recently, on wearables. (Note: Due to distinct licensing structures and specific technical implementation variations, the Android OS is typically categorized separately from standard Linux distributions). Furthermore, because the Linux kernel natively supports dozens of processor architectures, it serves as the primary choice for single-board computers like the Raspberry Pi.

¹According to data published by <https://www.top500.org> profiling the world’s top 500 supercomputers, 498 systems ran Linux in 2017. From June 2018 onward, every single supercomputer on the Top500 list runs on a Linux-based operating system.

²Data source: <https://statcounter.com>

1.3 Using Linux

1.3.1 The Command-Line Format

Modern Linux desktop distributions ship with sophisticated Graphical User Interfaces (GUIs) that mimic the functional design of Windows, with variations mostly limited to user habits. A standard user can perform the majority of daily workflows within this graphical paradigm. However, for software developers, relying entirely on a GUI is inefficient; furthermore, production target embedded systems frequently operate completely without graphical heads.

Linux features a robust text-based management environment known as the **Command-Line Interface** (CLI). Operating via the command line requires learning specific structural rules and memorizing key syntax utilities, which occasionally creates a false impression that “Linux is difficult to use.”

It is important to separate regular consumers from computer developers. Consumers need a plug-and-play OS where a GUI is sufficient; developers, however, require the command line because it offers unmatched flexibility and detailed control, allowing complex or automated tasks to be executed in ways that are cumbersome or impossible within a standard GUI.

The command-line interface is rendered through a window called a **terminal**. The core program inside the terminal is the **shell**, which reads keyboard input, calls the requested program, executes it, and prints the result. Typing a line of text next to the terminal prompt character (`$` or `#`) and pressing Enter creates an executable command statement line.

- The `$` symbol represents a regular user account prompt.
- The `#` symbol denotes the root superuser account prompt.

These prompt layouts can be customized by the user. Commands that modify core system settings are restricted and will fail if executed from a regular user prompt without elevated privileges.

A complete command-line entry consists of three distinct parts: the command, options, and arguments:

```
$ command [options] [arguments]
```

The ‘command’ specifies the action you want to perform; the ‘options’ modify **how** the action is executed; and the ‘arguments’ specify the target object of the operation.

The square brackets ‘[]’ signify that these components are optional depending on the utility; the brackets themselves are not typed. Some basic commands require no options or arguments, whereas others accept multiple items. Certain options require an adjacent argument directly following them.

Commands, options, and arguments must be separated by spaces. Options are typically prefixed with one dash ‘-’ (short options) or two dashes ‘--’ (long options), which form an unbroken string with the option character.

CRITICAL RULE: The Linux command line is strictly **case-sensitive**.

For example, to forcefully delete a Python file named ‘hello.py’, you would execute:

```
$ rm -f hello.py
```

In this statement, ‘rm’ (remove) is the core command, ‘-f’ (force) is the operational option, and ‘hello.py’ is the target file argument.

1.3.2 Users and Permissions

We can examine file systems, ownerships, and permissions using the ‘ls’ (list) directory utility. A frequent option used with this command is ‘-l’ (long listing format), which displays comprehensive file attributes. Another option, ‘-i’, displays the unique inode numbers of files. If the arguments do not conflict or cause syntax confusion, multiple short options can be combined behind a single dash (e.g., ‘-li’).

```
$ ls -li
4850428 drwxr-xr-x 2 pi pi      4096 Feb  8 11:35 graphics
4850423 -rw-r--r-- 2 pi pi 1524296 Feb  8 12:03 Pithon.pdf
4850317 -rwxr-xr-x 1 pi pi      5315 Feb  8 14:21 hello.py
4850423 -rw-r--r-- 2 pi pi 1524296 Feb  8 12:03 lecture.pdf
4850510 drwxr-xr-x 2 pi pi      4096 Feb  4 16:00 program
4850553 drwxr-xr-x 2 pi pi      4096 Feb  8 13:16 text
```

Each row in the terminal output details a single file block across 8 distinct fields:

1. inode number
2. File type & Access permissions
3. Hard link count
4. Owner user
5. Owner group

6. File size (in bytes)
7. Modification date/time
8. Filename

An **inode** (index node) is a data structure on a storage drive that describes the physical properties and allocation blocks of a file. Identical inode numbers indicate that separate directory entries point to the exact same physical storage data on the disk. This mechanism underpins Linux hard links. In the listing above, the third field indicates the hard link count. For instance, `'Python.pdf'` and `'lecture.pdf'` both share the inode number `'4850423'`. They are not duplicates; they are two different filenames pointing to the exact same file data on the disk.

The file type and permission string consists of 10 characters. The first character denotes the explicit file category, as detailed in Table 1.1.

Table 1.1: Linux File Type Identifiers

Symbol	File Type Category	Common Origin/Created via
–	Regular File	text editors, compilers, or applications
b	Block Device File	system initialization or the <code>'mknod'</code> utility
c	Character Device File	system initialization or the <code>'mknod'</code> utility
d	Directory	the <code>'mkdir'</code> command
l	Symbolic Link (Soft Link)	the <code>'ln -s'</code> command
p	Named Pipe (FIFO)	the <code>'mkfifo'</code> or <code>'mknod'</code> commands
s	Network Socket	the system network call <code>'bind()'</code>

The remaining 9 characters of the permission string are broken into three blocks of 3 characters each. These blocks define the permissions for the **Owner User**, the **Owner Group**, and all **Others**, respectively. The owner and group names are listed in the fourth and fifth fields.

Permissions within a block consist of read, write, and execute capabilities, represented by the characters `'r'`, `'w'`, and `'x'`. If a permission is withheld, its position is marked with a dash `'-'`.

In Linux, user configuration records are managed inside the `'/etc/passwd'` file. This file lists the username, User ID (UID), Group ID (GID), user metadata, and default shell login path for every user on the system. Historically, it also stored encrypted password strings. However, because this file must remain globally readable so processes

can map numeric IDs to names, storing passwords there created security risks from offline brute-force attacks. Consequently, password hashes are now securely isolated inside the `/etc/shadow` file, which can only be read with superuser root privileges.

Group allocations are handled via `/etc/group`, which records group names, Group IDs (GIDs), and user membership rosters. This file includes a group password field, though it is rarely used in modern systems.

Group partitioning in Linux simplifies project collaboration and serves as a fundamental security boundary. Every operation on the system is strictly bound to the permission limits of the executing user; any attempt to bypass these limits will be blocked by the kernel. The superuser account, named `root`, possesses absolute system administrative privileges. To prevent accidental system damage, administrators should avoid logging into a system directly as root for regular tasks.

1.3.3 Directory Operations

The primary command-line tools used to navigate and manipulate directories include:

- `mkdir` (make directory): Creates one or more new directories.

```
$ mkdir dir1 dir2 dir3
$ mkdir -p newdir/dir2
```

The `-p` (parents) option tells the utility to automatically generate any missing parent directories along the path instead of throwing an error.

- `rmdir` (remove directory): Deletes an empty directory.

```
$ rmdir dir1 newdir/dir2
```

This utility will fail if the target directory contains files or subdirectories, protecting users from accidental data loss.

- `cd` (change directory): Shifts the current working path of the shell.

```
$ cd /home/student/Desktop
$ cd ~
```

Running `cd` with no arguments automatically returns the shell to the user's home directory. The tilde symbol `~` can also be used as a shortcut for the home directory.

- **pwd** (print working directory): Outputs the absolute path of the current directory. It requires no arguments.

Linux uses the forward slash ‘/’ as a path separator. The topmost boundary of the file system layout is known as the **root directory** (‘/’). Within every directory on the system, two special relative entries exist: ‘.’ (a single dot) represents the current directory, while ‘..’ (two dots) points to the immediate parent directory.

1.3.4 File Operations

The primary command-line tools used to manage files include:

- **cp** (copy): Duplicates files or directories. To copy a single file:

```
$ cp [options] source_file target_file
```

If the target file already exists, it will be overwritten silently. To prevent accidental data loss, use the ‘-i’ (interactive) option, which prompts for confirmation before overwriting an existing file.

When copying multiple files, the final argument must be a directory. To copy an entire directory along with its contents, you must include the ‘-r’ (recursive) option; otherwise, directories will be skipped.

To simplify operations on groups of files with similar names, Linux supports **wildcards**. Table 1.2 lists the most common wildcards used in file operations.

For example, to recursively copy every file and folder inside your home directory to the ‘/mnt’ directory while prompting before overwriting, execute:

```
$ cp -ri ~/* /mnt
```

- **mv** (move): Relocates or renames files and folders. It can rename a file by moving it to a new filename, or move multiple files into a destination directory. Similar to ‘cp’, adding the ‘-i’ option prompts before overwriting files.
- **rm** (remove): Deletes files permanently. Unlike modern desktop environments that safely move files to a temporary “Trash” folder, the ‘rm’ command bypasses recovery mechanisms and unlinks data immediately, freeing disk space. Using the ‘-i’ option prompts for confirmation before deleting each file. The shell will also prompt you if you attempt to delete a read-only file.

Table 1.2: Common File Management Wildcards

Wildcard	Functional Meaning	Syntax Example & Meaning
<code>?</code>	Matches exactly one arbitrary character	<code>'???'</code> (Matches filenames exactly three characters long)
<code>*</code>	Matches zero or more characters (except leading dots)	<code>'*.c'</code> (Matches all files ending with a <code>'c'</code> extension)
<code>[]</code>	Matches any single character enclosed in the brackets	<code>'*[Aa]'</code> (Matches names ending in <code>'A'</code> or <code>'a'</code>)
<code>[-]</code>	Matches a range of consecutive characters	<code>'[A-Z]*'</code> (Matches any name starting with an uppercase letter)
<code>^</code> or <code>!</code>	Matches any character not listed inside the brackets	<code>'[^0-9]*'</code> (Matches filenames that do not begin with a digit)
<code>{ }</code>	Matches any comma-separated string within the braces	<code>'*.c.{[a-z]*}'</code> (Matches target extensions or string blocks)

```
$ ls -l file?
-rw-rw-r-- 1 pi pi 0 Feb 20 08:44 file1
-r--r--r-- 1 pi pi 0 Feb 20 08:44 file2
$ rm file?
rm: remove write-protected regular empty file 'file2'? y
```

To bypass these confirmation prompts during automated operations, use the `'-f'` (force) option. To delete directories along with all their nested contents, use the `'-r'` (recursive) option.

WARNING: Files deleted via the `'rm'` command cannot be recovered through regular system tools.

- File Viewing Utilities

While text editors can display file contents, they risk accidental modifications and load large files slowly. For read-only viewing, use these dedicated utilities instead:

1. `cat` (concatenate): Prints the entire file contents directly to the terminal

screen. For long files, use ‘Shift + PageUp’ or ‘Shift + PageDown’ to scroll through the terminal buffer. Note that terminal screens have fixed buffers (typically 1,000 to 2,000 lines), meaning older lines will be dropped.

2. **more/less** (paging utilities): Display file contents one screenful at a time, pausing for keyboard input to advance or scroll. ‘less’ is generally preferred as it allows both forward and backward navigation.
3. **head/tail** (boundary viewers): By default, ‘head’ outputs the first 10 lines of a file, while ‘tail’ outputs the final 10 lines. Use the ‘-n’ option followed by a number to change the line count. Combined with a Linux pipeline, these tools can extract a specific section from the middle of a file:

```
$ head -n 20 car.py | tail -n 5
```

The vertical bar ‘|’ is the **pipe operator**. It redirects the standard output streams of the left-hand command into the standard input stream of the right-hand command. The example above pipes the first 20 lines of ‘car.py’ into ‘tail’, which prints the final 5 lines, effectively isolating lines 16 through 20.

4. **tac** (reverse cat): Operates exactly like ‘cat’, but reverses the output line order to print from bottom to top. It is commonly used to inspect system logs, as recent entries are appended to the end of the file.
- **chmod** (change mode): Modifies file permissions. The permission attributes shown by ‘ls -l’ (e.g., ‘-rwxr-xr-x’) can be modified using octal values or symbolic notation, except for the first character which defines the file type.

The three permission blocks (User, Group, Others) can be represented as a three-digit octal number, where Read=4, Write=2, and Execute=1. For example, to give the owner full permissions (7) and everyone else read and execute permissions (5), use:

```
$ chmod 755 hello.py
```

Alternatively, you can use symbolic notation with target groups (‘u’ for user, ‘g’ for group, ‘o’ for others, ‘a’ for all), operators (‘+’ to add, ‘-’ to remove, ‘=’ to set explicitly), and permission flags (‘r’, ‘w’, ‘x’). To make a script executable for everyone, use:

```
$ chmod a+x hello.py
```

Unlike Windows, which relies on file extensions (like `.exe` or `.com`) to identify executables, Linux relies strictly on these permission bits to determine whether a file can be executed as a program.

- **tar** (tape archive): Bundles multiple files into a single archive for storage or transfer, often compressing the output at the same time. Table 1.3 lists its common options.

Table 1.3: Common ‘tar’ Command Options

Option	Functional Meaning	Option	Functional Meaning
-A	Append tariff archives to an existing archive	-t	List the contents of an archive file
-c	Create a brand new archive file	-x	Extract files out from an archive bundle
-d	Diff check archive against active file system	-X	Exclude files listed in a separate file
-u	Update older files within an existing archive	-f	Specify the target archive filename
--delete	Delete specific member files from an archive	-j	Compress or decompress using the ‘bzip2’ algorithm
-r	Append regular files to the tail of an archive	-J	Compress or decompress using the ‘xz’ algorithm
-v	Verbose: display processing status filenames	-z	Compress or decompress using the ‘gzip’ algorithm
-C	Change to a specified directory before running		

Note: In traditional UNIX syntax, the leading dash ‘-’ for ‘tar’ options can be omitted. A standard operation must include exactly one primary action flag (‘c’, ‘x’, ‘t’, ‘A’, ‘d’, ‘r’, or ‘u’).

To compress and bundle a folder named `Desktop` using the ‘bzip2’ algorithm into an archive named `desktop.tar.bz2`, run:

```
$ tar cjf desktop.tar.bz2 Desktop/
```

When extracting archives, ‘tar’ detects the compression format automatically. Specifying an incorrect decompression flag manually will cause the command to

fail. To extract an archive into a specific directory, use:

```
$ tar xvf desktop.tar.bz2 -C Downloads/
```

This extracts the contents of ‘desktop.tar.bz2’ into the ‘Downloads/’ folder while listing the files on screen. You can also use standalone compression utilities like ‘gzip’/‘gunzip’ or ‘zip’/‘unzip’. Different compression algorithms offer varying trade-offs between compression ratio, compression speed, and decompression times.

1.3.5 Process Control

A **process** is an active, running instance of a program managed by the operating system as a single unit of work. While a simple tool runs as a single process, complex applications often spawn multiple processes.

To view active system processes, use the ‘ps’ (process status) command. The ‘ax’ options list all running processes across the system:

```
$ ps ax
  PID TTY          STAT TIME  COMMAND
    1 ?           Ss    1:45  /sbin/init splash
    2 ?           S      0:00  [kthreadd]
    4 ?           S<    0:00  [kworker/0:0H]
20755 ?           Ssl    0:00  /usr/lib/evince/evince
21326 ?           Ss    0:00  /lib/systemd/systemd --user
21327 ?           S      0:00  (sd-pam)
23376 pts/0        Sl    0:47  evince LinuxBasis.pdf
23885 pts/2        S      0:01  bash
25034 pts/0        S+    3:20  vim text/commandline.tex
```

PID (Process ID): A unique 16-bit positive integer assigned sequentially by the kernel to track each process. Process ‘1’ is the system initialization daemon (‘init’ or ‘systemd’). If the maximum PID limit (32,767 by default) is reached, the counter wraps around to assign available lower numbers.

TTY: Specifies the terminal controlling the process. Core system daemons and graphical interfaces lack an associated terminal and display a ‘?’.

STAT: Indicates the current execution state of the process. A process exists in one of four primary states: Running (‘R’), Sleeping/Waiting (‘S’), Stopped (‘T’), or Zombie/Defunct (‘Z’). These states can include sub-flags (e.g., ‘s’ for session leader, ‘+’ for foreground process).

1.3.6 Modifying Process States

You can terminate or signal running processes using the ‘kill’, ‘pkill’, or ‘killall’ commands. The ‘kill’ command targets processes by their numeric PID, whereas ‘pkill’ and ‘killall’ target processes by their alphanumeric “program name”:

```
$ kill 23376          # Targets process by numeric PID
$ pkill evince        # Targets process by executable name
```

Despite its name, the ‘kill’ utility does not destroy processes directly; instead, it sends an asynchronous **signal** to them. Signals are a core mechanism for inter-process communication (IPC) in Linux. By default, ‘kill’ sends a ‘SIGTERM’ (15, terminate) signal, which requests the target process to save its state and exit cleanly.

Table 1.4 lists common Linux signals. You can specify these signals using either their numeric value or symbolic name as an option to the ‘kill’ command.

Some processes may hang or ignore a standard ‘SIGTERM’ signal. In these scenarios, you can force termination by sending a non-catchable ‘SIGKILL’ signal using the ‘-9’ option:

```
$ kill -9 23376
```

Signals can also control execution states without terminating the process. For example, you can pause and resume a process using ‘SIGSTOP’ and ‘SIGCONT’:

```
$ kill -STOP 23376    # Pauses the target process
$ kill -CONT 23376    # Resumes the paused process
```

1.3.7 Foreground and Background Execution

When launching graphical applications or long-running tasks from a terminal, you can append an ampersand symbol ‘&’ to the end of the command line. This runs the program as a **background process**, leaving the terminal prompt free to accept subsequent commands immediately:

```
$ idle3 &
```

You can manage these background tasks using the shell’s built-in ‘fg’ (foreground) and ‘bg’ (background) utilities. Running ‘fg’ brings the most recent background task back into the foreground.

If multiple jobs are running concurrently in the background, use the built-in ‘jobs’ command to list them along with their index numbers. You can pass these index numbers to ‘fg’ to target a specific job:

Table 1.4: Standard Linux System Signals

Signal Name	Numeric Value	Description/Operational Behavior
SIGHUP	1	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Interrupt from keyboard (triggered via ‘Ctrl + C’)
SIGQUIT	3	Quit from keyboard; forces a core dump execution
SIGABRT	6	Abort signal; indicates abnormal terminal execution
SIGKILL	9	Kill signal; forces immediate termination (cannot be blocked or caught)
SIGUSR1	10	User-defined signal 1; available for custom application actions
SIGSEGV	11	Segmentation fault; invalid or restricted memory access reference
SIGUSR2	12	User-defined signal 2; available for custom application actions
SIGPIPE	13	Broken pipe; write executed to a pipe with no readers
SIGALRM	14	Timer signal emitted by the kernel alert clock
SIGTERM	15	Termination signal; requests a clean program exit (default action)
SIGCHLD	17	Child process stopped, terminated, or changed state
SIGCONT	18	Continue execution if currently stopped
SIGSTOP	19	Stop process execution immediately (cannot be blocked or caught)

```
$ fg %1
```

Pressing ‘Ctrl+Z’ in the terminal sends a ‘SIGSTOP’ signal to the active foreground process, pausing it and returning you to the shell prompt. You can then run ‘bg’ to let the paused process resume execution in the background, or ‘fg’ to resume it in the foreground.

1.3.8 I/O Redirection and Pipelines

Standard Input/Output (I/O) redirection is a powerful feature of the Linux file system architecture. At the system level, all user interactions map to file reads and writes. Redirection allows you to change where a program reads its input from or writes its output to.

Table 1.5 lists the standard redirection operators.

Table 1.5: Standard Shell I/O Redirection Operators

Operator Symbol	Functional Redirection Meaning
>	Redirects standard output (stdout) to a file (overwrites existing file contents)
>>	Redirects standard output (stdout) to a file (appends to existing file contents)
<	Redirects standard input (stdin) from a file
<< Delimiter	Here-doc: Feeds input into a command until the specified delimiter string is met
<<< string	Here-string: Passes an explicit inline text string directly to standard input

To extract lines 16 through 20 from ‘hello.py’ and write them into a new file named ‘hello_bak.py’, combine a pipeline with output redirection:

```
$ head -n 20 hello.py | tail -n 5 > hello_bak.py
```

To append a line of text to the end of an existing file without modifying its current contents, use the append operator ‘>>’:

```
$ echo "hello, world" >> hello.py
```

IMPORTANT: The single ‘>’ operator creates the target file if it does not exist, but **overwrites** it silently if it does. The double ‘>>’ operator appends data safely to the end of the file.

If a script requires automated keyboard input, you can pre-write the responses into a text file and pass it to the script using input redirection (‘<’), avoiding the need for manual interaction:

```
$ head < hello.py
```

1.4 Text Editors

Writing code requires editing plain-text files without hidden formatting codes. In the Linux ecosystem, **VIM** (Vi Improved) is one of the most widely used and powerful command-line text editors.

VIM runs directly inside the terminal without standard menus or mouse interactions, relying entirely on keyboard shortcuts. It operates using three primary modes, as illustrated in Figure 1.3:

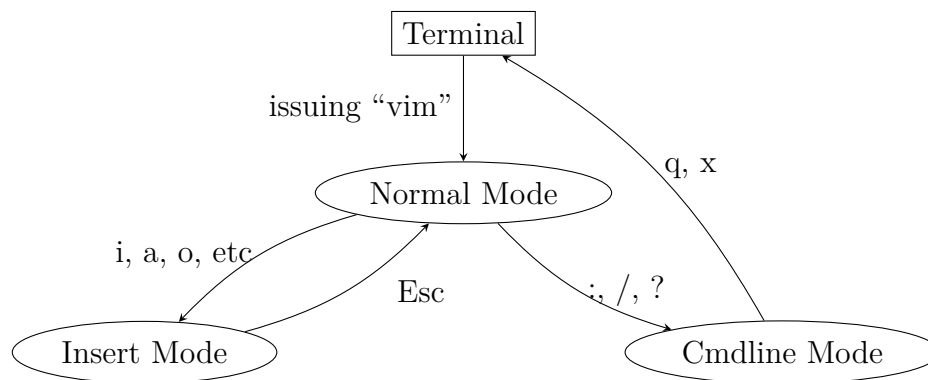


Figure 1.3: VIM Mode Transitions

1. **Normal Mode (Command Mode):** The default landing mode when opening a file. Keypresses are interpreted as navigation or structural editing commands. Typing insertion keys (such as ‘a’ or ‘i’) switches the editor to Insert Mode. Typing punctuation keys (such as ‘:’, ‘/’, or ‘?’) switches it to Command-Line Mode.
2. **Insert Mode (Editing Mode):** Functions like a conventional text editor where typing inputs characters and arrow keys move the cursor. Pressing the ‘Esc’ key exits Insert Mode and returns you to Normal Mode.
3. **Command-Line Mode (Ex Mode):** Activates a prompt at the very bottom line of the screen to execute global operations, such as saving, quitting, or running advanced regex search-and-replace actions.

VIM Command Reference

To switch from Normal Mode into Insert Mode, press one of the following keys:

- i/I: Insert text before the cursor/at the beginning of the current line.
- a/A: Append text after the cursor/at the end of the current line.

- o/O: Open a new blank line below/above the current line.
- R: Enter replace mode to overwrite text characters.
- s/S: Substitute a single character/the entire current line.
- C: Delete text from the current cursor position to the end of the line and enter Insert Mode.

Common editing shortcuts in Normal Mode:

- ~: Toggle character capitalization between uppercase and lowercase.
- x/X: Delete the character directly under/before the cursor.
- dd/D: Delete (cut) the entire current line/from the cursor to the end of the line.
- dt.: Delete text from the current cursor position up to the first period ‘.’ character.
- J: Join the current line with the line directly below it.
- u/Ctrl+R: Undo the last action/Redo the undone action.
- .: Repeat the immediate preceding editing operation.
- y/yy: Copy (yank) the current visual selection/the entire current line into the buffer.
- p/P: Paste the buffer contents directly after/before the current cursor position.
- ZZ: Save all changes and exit VIM immediately.

Efficient cursor navigation in Normal Mode:

- 0/\$: Move the cursor to the exact start/end of the current line.
- ^: Move the cursor to the first non-whitespace character on the line.
- tx/fx: Jump the cursor right up to/onto the next occurrence of character ‘x’.
- 5h/5j/5k/5l: Move 5 characters left, 5 lines down, 5 lines up, or 5 characters right.
- 100G: Jump the cursor directly to line number 100.

Common actions in Command-Line Mode (prefixed with a colon ‘:’):

- :w file: Save (write) changes out to a specific file target.

- `:q/:q!`: Quit the editor/Force-quit immediately, discarding all unsaved modifications.
- `:x`: Save changes and quit (equivalent to running `‘w’` followed by `‘q’`).
- `:1,20s/foo/bar/g`: Substitute every instance of string `‘foo’` with `‘bar’` between lines 1 and 20.
- `:%s/foo/bar/g`: Substitute every instance of string `‘foo’` with `‘bar’` across the entire file.
- `.,$-20s/foo/bar/g`: Substitute strings from the current line (`‘.’`) up to 20 lines from the end of the file (`‘$-20’`).
- `:1,20d`: Delete lines 1 through 20.
- `:5,10m3`: Move lines 5 through 10 and insert them directly after line 3.
- `:g/for/l`: Globally search and list every line containing the string match `‘for’`.
- `:v/for/d`: Invert match: globally delete all lines that **do not** contain the string `‘for’`.

Advanced VIM Layout Features:

- Window Splitting: In Normal Mode, press `‘Ctrl + W’` followed by `‘s’` to split the screen horizontally, or `‘v’` to split it vertically. This allows you to view different files simultaneously or look at two separate sections of the same file side-by-side.
- Visual Block Mode: Press `‘Ctrl + V’` to select text blocks by column rather than full rows. This allows you to delete columns of text, move blocks, or insert characters across multiple rows simultaneously (by typing `‘Ctrl + V’`, selecting rows, pressing `‘I’`, typing text, and hitting `‘Esc’`).

For beginners, VIM includes a built-in interactive tutorial called `‘vimtutor’`. You can access the complete documentation at any time inside VIM by typing `‘:help’`.

1.5 Network Connectivity

1.5.1 Remote Terminal Access

In embedded applications, target single-board computers like the Raspberry Pi are typically deployed as headless units without dedicated keyboards, mice, or monitors.

While the Raspberry Pi natively supports standard USB devices and HDMI outputs, a dedicated workstation setup is usually unnecessary during development. Instead, developers can manage the hardware remotely over a local network.

Most Linux distributions run a background **SSH** (Secure Shell) service to provide secure command-line access over a network. Suppose your Raspberry Pi has an IP address of ‘192.168.2.103’.³ When it runs the SSH daemon (‘sshd’) and is connected to your local network, you can log in remotely from your personal computer’s terminal:

```
$ ssh pi@192.168.2.103
The authenticity of host '192.168.2.103 (192.168.2.103)' can't be established.
ECDSA key fingerprint is SHA256:rrFsuRrEhzNaGea15HNvGRCcYDJmaCbAwMYmja+fyD0.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.2.103' (ECDSA) to the list of known hosts.
pi@192.168.2.103's password:
```

During your first connection, SSH displays a security warning verifying the remote host’s fingerprint. Type ‘yes’ to save the host key to your local configuration. For subsequent connections, you will only be prompted for the user account password. Once authenticated, the terminal displays the remote login banner:

```
#####
# Raspberry Pi3 (aarch64)                #
# Desktop                                #
# WELCOME!                               #
#####
pi@raspberrypi:~$
```

In this example, the Raspberry Pi’s network IP address is ‘192.168.2.103’ and the login user is ‘pi’. For security reasons, most Linux distributions block remote root logins by default. Once logged in, any commands you type run directly on the Raspberry Pi. This terminal environment functions exactly like a local shell, missing only the graphical desktop.

The SSH suite also includes ‘scp’ (secure copy) for secure file transfers over the network:

```
$ scp file1 user@192.168.200.170:file2
```

³An Internet Protocol address (IP address) is a numerical label, such as 192.168.1.1 (IPv4) or 2001:0db8:0000:0000:8a2e:0370:7334 (IPv6), that is assigned to a device connected to a computer network that uses the Internet Protocol for communication. IP addresses serve two main functions: network interface identification, and location addressing.

This copies the local file ‘file1’ to the home directory of the user ‘user’ on the remote machine ‘192.168.200.170’, renaming it ‘file2’. You will be prompted to enter the remote user’s password. If you want to keep the original filename, omit the target filename but keep the trailing colon:

```
$ scp user@192.168.200.170:file1 .
```

The dot ‘.’ represents the current local directory. This copies ‘file1’ from the remote machine directly into your local working directory under its original name.

Configuring Passwordless SSH Key Authentication

Entering passwords for every command or file transfer can slow down development. You can configure key-based authentication to log in securely without typing passwords:

1. Generate an SSH key pair on your development machine:

```
$ ssh-keygen -t rsa -b 4096
```

The ‘-t’ option specifies the encryption algorithm, and ‘-b’ defines the key length in bits. The utility generates a private key (‘id_rsa’) and a public key (‘id_rsa.pub’) inside your ‘~/ .ssh/’ folder. You can add an optional passphrase to encrypt the private key for extra security.

SECURITY RULE: The public key can be shared freely, but your private key must remain secure on your host machine.

2. Copy the public key to the remote Raspberry Pi:

```
$ ssh-copy-id pi@192.168.2.103
```

This automated tool uploads your local public key (‘id_rsa.pub’) and appends it to the ‘~/ .ssh/authorized_keys’ file on the remote machine, performing an operation equivalent to:

```
pi@remote:~$ cat id_rsa.pub >> ~/.ssh/authorized_keys
```

Once configured, you can run remote logins and ‘scp’ transfers instantly without password prompts.

1.5.2 Remote Desktop Access

If your development work requires a graphical interface, you can pass the ‘-X’ option through SSH to enable **X11 Forwarding**. This forwards graphical windows from the remote server to your local display:

```
$ ssh pi@192.168.2.103 -X
```

When you launch a GUI application from this session, its interface renders inside a local window on your desktop. This requires X11 forwarding to be enabled in the remote server's configuration file (`/etc/ssh/sshd_config`):

```
X11Forwarding yes
```

Alternatively, you can use **VNC** (Virtual Network Computing) for a full remote desktop experience. To start a VNC server on the Raspberry Pi, use `'x11vnc'`:

```
# x11vnc -display :0 -shared -forever >/dev/null 2>/dev/null &
```

- `-display :0`: Connects to the primary local desktop display session.
- `-shared`: Allows multiple client connections simultaneously.
- `-forever`: Keeps the VNC server running after a client disconnects.
- `>/dev/null 2>/dev/null&`: Suppresses output messages and runs the server in the background.

Once the VNC server is running, connect from your host machine using a client viewer like `'vncviewer'`:

```
$ vncviewer 192.168.2.103
```

This opens a window displaying the active remote desktop environment (such as the lightweight XFCE4 desktop shown in Figure 1.4).



Figure 1.4: XFCE4 Graphical Desktop Environment Session on a Raspberry Pi

1.5.3 Remote Network Storage (NFS)

Embedded devices often have limited onboard storage. You can expand available space using a **Network File System (NFS)** to share directories from your development PC with the Raspberry Pi.

First, configure your PC as an NFS server by adding the shared directory paths and access permissions to `/etc/exports`:

```
/srv/nfs4 192.168.*.*(rw,sync,subtree_check,no_root_squash)
```

Assuming your PC's network IP address is `192.168.200.170`, log into the Raspberry Pi as root and mount the shared directory onto a local folder path:

```
# mount 192.168.200.170:/srv/nfs4 /mnt -o nolock,proto=tcp
```

This mounts the PC's `/srv/nfs4` directory directly onto the Raspberry Pi's `/mnt` folder. Both machines now access the exact same files simultaneously, while the physical data remains stored on the PC's hard drive. The `-o proto=tcp` option forces the connection to use the TCP protocol, avoiding the data corruption or performance drops that can occur with NFS's default UDP transport when copying large files.

Using network connections during embedded development eliminates the need for extra hardware peripherals like spare keyboards and monitors. It also lets you leverage your host computer's storage capacity and processing power to streamline your development workflow.

1.6 Chapter Summary

This chapter covered fundamental operating system architecture concepts, core features of the Linux operating system, and standard terminal workflows. Review and master the essential commands summarized in Table 1.6 below.

Table 1.6: Summary of Core Linux Commands

Command Name	Core System Function	Common Practice Option Flags
ls	Lists files and subdirectory entries	'-l' (long format), '-a' (show hidden files), '-i' (show inodes)
more/less	Displays text file contents page by page	Navigational keys, text search parameters
cp	Copies target files or folder pathways	'-r' (recursive directory copy), '-i' (interactive prompt)
cd	Changes the shell's active working directory	Target path argument (defaults to home directory '~')
mv	Moves or renames files and directories	'-i' (interactive alert prompts)
mkdir	Generates brand new directory structures	'-p' (automatically creates missing parent folders)
rmdir	Deletes empty target directories	Only executes successfully if the directory is completely empty
rm	Deletes target files permanently	'-i' (interactive prompt), '-f' (force), '-r' (recursive purge)
ps	Displays active system execution processes	'ax' (lists all system processes), 'aux' (includes user details)
tar	Bundles, compresses, and extracts archives	'-c' (create), '-x' (extract), '-v' (verbose), '-f' (archive file)

Chapter 2

Very Short Python Introduction

2.1 Introduction

Python is an interpreted, high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. Van Rossum developed the language while working as a researcher at CWI (Centrum Wiskunde & Informatica, English: Center for Mathematics and Computer Science) in the Netherlands. In December 1989, van Rossum was looking for a “hobby programming project that would keep him occupied during the week around Christmas” when his office was closed.

Python’s name is derived from the British comedy group *Monty Python’s Flying Circus*, which Guido van Rossum enjoyed watching while developing the language.



Guido van Rossum

Python is a multi-paradigm programming language. Object-oriented programming and structured programming are natively supported, and many of its features facilitate functional programming and aspect-oriented programming. Many other development paradigms are supported via ecosystem extensions, including design-by-contract and logic programming.

Large-scale organizations that rely heavily on Python include Wikipedia, Google, Yahoo!, CERN (French abbreviation for the European Organization for Nuclear Research), NASA (National Aeronautics and Space Administration), Facebook, Amazon, Instagram, and Spotify. The social news and aggregation platform Reddit is written almost entirely in Python. Additionally, Python has been successfully embedded into numerous commercial and open-source software products as a robust scripting runtime.

In modern Linux environments, a large portion of system administration tools and application software packages are written entirely in Python. Reflecting this massive growth, Python received the prestigious TIOBE Programming Language of the Year award in 2007, 2010, 2018, and 2020—an honor granted to the language demonstrating the highest annual popularity growth momentum.

2.2 Python Environment

The official Python website (<https://www.python.org/downloads>) provides pre-compiled installation binaries for all mainstream operating systems. Alternatively, developers can download the complete source code to configure, modify, or compile the interpreter locally. Building from source can be a resource-intensive and time-consuming process; it is generally unnecessary unless you are tailoring the runtime for a custom, highly constrained embedded system. Because Python is fundamentally cross-platform software, its operational behavior is consistent regardless of the underlying operating system architecture.

Most program development requires specialized tooling, and Python is no exception. However, because Python’s syntax design is highly minimal and clean, a well-configured text editor can easily serve as an Integrated Development Environment (IDE). Python is an essential dependency within modern Linux ecosystems; consequently, the vast majority of Linux distributions ship with a Python interpreter installed by default.

The standard development environment bundled with Python on Linux is IDLE (*Integrated Development and Learning Environment*), which is itself written entirely in Python using the `tkinter` GUI toolkit. Its core wrapper script contains only a few initialization lines:

Listing 2.1: `idle3`, Python’s Default IDE Wrapper

```
#!/usr/bin/python3

from idlelib.PyShell import main
if __name__ == '__main__':
    main()
```

This environment includes both a Python interactive shell and a script editor that features full Python syntax highlighting. Even without installing IDLE, developers can invoke the interactive interpreter directly from any standard Linux command shell by executing the `python3` binary. This interactive shell is ideal for learning the language

syntax, inspecting APIs, and conducting rapid micro-benchmarks. However, it is not practical for building complex applications, as code modifications are cumbersome and the entered lines cannot be saved directly to a persistent file for later execution.

2.3 Python Basics

2.3.1 Running Your First Python Program

The traditional entry point for learning Python is the classic “Hello, world!” script:

```
#!/usr/bin/python3
print("Hello, world!")
```

This minimal script contains only two lines. When writing scripts, ensure that no accidental leading whitespaces exist at the beginning of these execution lines.

To execute this source code on a computer, it must first be saved into a file, such as `hello.py`. The `.py` extension explicitly designates it as a Python source file. On Windows systems, double-clicking this file tells the operating system shell to pass it to the registered Python interpreter binary. On Linux and other UNIX-like operating systems (such as macOS), there are three primary methodologies for running a Python program:

1. Explicit Interpreter Invocation via the Command Line:

```
$ python3 hello.py
```

The interpreter parses the script, executes the commands sequentially, and prints `Hello, world!` directly to the standard output stream of the terminal.

2. Direct Script Execution via Executable Permissions:

First, modify the file’s access permissions to grant execute attributes using the system utility:

```
$ chmod +x hello.py
```

Once the executable bit is set, the script can be invoked directly by specifying its relative path under the current working directory:

```
$ ./hello.py
```

The prefix `./` explicitly denotes the current directory. When executed this way, the Linux program loader reads the first line of the script. The special `#!` character

sequence (known as a **shebang**) indicates that the rest of the line specifies the path to the interpreter binary responsible for parsing the file.

3. **Graphical Execution:** If a desktop environment is active on the Linux target, the file can be executed by clicking its icon within the file manager, provided the file manager is configured to run Python scripts automatically.

In Python source files, a single physical line can host multiple independent statements if they are explicitly separated by a semicolon (;). The trailing semicolon at the absolute end of a line is optional. However, placing multiple statements on a single line is discouraged by Python style guidelines unless the individual operations are exceptionally concise.

Unlike many popular programming languages that use curly braces ({}) or keyword blocks to isolate execution scopes, Python relies strictly on leading whitespace indentation to delimit block structures. An increase in leading indentation denotes the beginning of a nested block, whereas a decrease back to the matching column alignment signifies the closure of the current scope block. Consequently, a Python program's visual arrangement directly represents its logical and semantic execution structure.

2.3.2 Interactive Mode

Executing `python3` or appending an ampersand to launch IDLE in the background (`idle3 &`) opens an interactive terminal session. When the standard primary prompt (`>>>`) appears, you can enter commands to be executed immediately:

```
Python 3.8.2 (default, Jul 20 2020, 22:30:15)
[GCC 9.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, Python!")
Hello, Python!
>>>
```

While the interactive shell does not replace a structured script development environment, it serves as an excellent utility for testing small logic snippets, exploring module exports, and reading documentation. The built-in `help()` function allows developers to inspect the interactive reference manuals for any core keyword, function, or loaded software module.

IDLE acts as a lightweight graphical wrapper for this loop. Developers can open a fresh, dedicated script file editor via the menu navigation path **File** → **New File**, write their

source code, and execute the complete file within the shell by pressing the F5 shortcut or choosing Run → Run Module.

2.3.3 Numerical Variables

Constants

Python handles three distinct classifications of numeric constants: integers (**int**), floating-point decimals (**float**), and complex mathematical structures (**complex**). For example, 2 is processed natively as an integer; 2.5 and 2. are interpreted as floating-point numbers; and 2j or 2.5J represent imaginary numerical constants.

- Arithmetic operations (addition, subtraction, and multiplication) executed entirely between integers produce an integer result. However, division behavior is split into two specialized operators: floating-point division (/), which always evaluates to a **float**, and floor division (//), which truncates the fractional portion to return an integer value.
- The imaginary unit suffix symbol (j or J) must immediately follow the numeric coefficient. No multiplication operator (*) can be placed between the coefficient and the suffix, because an isolated j is interpreted by the parser as a standard variable name identifier.
- Python uses the double asterisk symbol (**) as its native exponentiation operator.

Consider the following interactive command stream examples:

```
>>> 12345678 + 87654321
99999999
>>> 1000 / 330
3.0303030303030303
>>> (1 + 2j) * (3 - 2j)
(7+4j)
>>> 2.8 // 0.03
93.0
>>> (1 + j) * (1 - j)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'j' is not defined
>>> (-2) ** 0.5
(8.659560562354934e-17+1.4142135623730951j)
```

Python also natively converts alternative numerical radix bases. Constants prefixed with `0b`, `0o`, or `0x` (case-insensitive) represent binary, octal, and hexadecimal values, respectively:

```
>>> 0xff - 0b1111
240
```

Variables

A variable is a symbolic name or reference pointing to an underlying data object. Python does not require explicit variable type declarations; variables are bound dynamically when an assignment statement (the single equals sign operator, `=`) is evaluated. Accessing an unassigned variable identifier will halt execution and raise a `NameError`:

```
>>> a1 = 10
>>> a2 = a2 + a1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a2' is not defined
>>> a2 = 20
>>> a2 = a2 + a1
>>> a1, a2
(10, 30)
```

Variable identifiers can consist of uppercase and lowercase letters, numeric digits, and underscores (`_`). However, an identifier cannot begin with a numeric digit; hence, `Zhang3` is a valid variable name, whereas `3Zhang` is syntactically invalid. Python 3 permits the use of arbitrary Unicode characters (such as Chinese characters) for variable identifiers, though this practice is generally discouraged in professional software engineering.

```
>>> 张三的年龄 = 32
>>> 李四 = 张三的年龄 + 3
>>> print("李四的年龄是 ", 李四)
李四的年龄是  35
```

Python also supports elegant sequence unpacking syntax, making object state swapping highly intuitive without requiring temporary variables:

```
>>> a, b = 200, 'hello'
>>> b, a = a, b
>>> print('a=%s,' % a, 'b=%d' % b)
```

```
a=hello, b=200
```

In Python, the percent token (%) serves multiple contextual syntactic duties. When placed between string templates and matching element data variables, it performs classic C-style printf string interpolation formatting:

```
>>> "%4x" % 123
'   7b'          # Hexadecimal string, right-aligned with space padding
>>> "%04X" % 456
'01C8'          # Hexadecimal string, padded with leading zeros
```

Table 2.1: String Format Specifiers in Python

Specifier	Target Data Type Representation
%s	String conversion representation
%d, %i	Signed decimal integer formats
%u	Unsigned decimal integer formats
%o	Octal integer format
%x, %X	Hexadecimal integer format (lowercase/uppercase)
%e, %E	Scientific exponential floating-point representation
%f, %F	Standard decimal floating-point formatting

When applied directly between two numerical values, the % operator calculates the arithmetic modulo remainder (e.g., `5 % 3` returns 2).

2.3.4 Strings

String variables store arbitrary textual data and sequence characters.

```
>>> greeting = "Hello, Python!"
>>> print(greeting)
Hello, Python!
>>> number = 35
>>> number + number
70
>>> string = "35"
>>> string * 3
'353535'
```

Strings can be wrapped interchangeably in single quotes (' approach) or double quotes (" approach). Triple quotes (""" or ''') are used to handle multi-line string blocks or text sections that span multiple document line breaks. Special control characters embedded inside strings can be escaped using a leading backslash (\) character:

```
>>> "\"Hello, world!\" she said"
'Hello, world!' she said'
```

Python provides specialized string literals using string prefixes, including raw strings (r), legacy Unicode strings (u), and byte arrays (b):

```
>>> rStr = r"Python is easy."
>>> uStr = u'Python isn\'t hard to learn.'
>>> bStr = b'Hello!'
```

When dealing with plain ASCII text characters, standard strings and explicit Unicode representations are practically identical. However, when building systems that use non-English character sets (such as Chinese), developers must carefully manage text encodings. Modern Python 3 handles all standard string symbols as internal Unicode. To prevent configuration or decoding errors when script files are processed across different operating systems, we highly recommend inserting the following two header lines at the very top of your Python source files:

```
#!/usr/bin/python3
# -*- coding: utf-8 -*-
```

While the standard hash character (#) denotes a code comment, these specific opening lines tell the operating system loader which interpreter binary to use, and tell the Python compiler to parse the source file using the UTF-8 character encoding format. Mixed or mismatched character encodings will cause data corruption or lead to runtime errors during text output.

Crucially, Python strings are fundamentally **immutable**. Once created, individual character positions inside a string object cannot be modified directly; modifying a string requires generating a new string object through re-assignment or manipulation functions:

```
>>> my_str = 'hello'
>>> print(my_str[0])
h
>>> my_str[0] = 'H'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

2.3.5 Boolean Logic

The Boolean primitive data type handles logical states and contains only two distinct values: `True` and `False`. In conditional evaluations, Python interprets the following values as falsy (evaluating to `False`):

1. Numerical zero structures of any numeric type (0, 0.0, 0j).
2. Strings with a length of zero ("" or '').
3. Empty collection sequences (empty dictionaries {}, sets `set()`, lists [], or tuples ()).
4. The special empty single-instance placeholder object: `None`.
5. The primitive constant value `False` itself.

All other structural instances or valid objects evaluate to `True`. Both logical states belong explicitly to the foundational core class `bool`:

```
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
```

The special value `None` represents the absence of a value or a null pointer state. When a function completes execution without encountering an explicit `return` statement, it returns `None` to the caller. Boolean expressions are primarily used in comparison operations, logical combinations, and conditional control flows:

```
>>> x = 123
>>> if x > 0:
...     print("x is greater than zero.")
x is greater than zero.
>>>
```

Python provides a complete set of relational and comparison operators:

Operator Flag	Computational Condition and Operational Purpose
<code>==</code>	Evaluates whether the values of two objects are equal
<code>!=</code>	Evaluates whether the values of two objects are unequal
<code>></code>	Strict greater-than comparison
<code><</code>	Strict less-than comparison
<code>>=</code>	Greater-than-or-equal-to comparison
<code><=</code>	Less-than-or-equal-to comparison
<code>is</code>	Evaluates object identity (checks if both variables reference the exact same memory address)
<code>is not</code>	Evaluates if two variables reference distinct underlying objects

Complex conditional logic can be constructed using the three text-based operators: **and**, **or**, and **not**. Their evaluation behaviors follow standard mathematical logic rules. For example, the statement:

```
x > 0 and x < 10
```

evaluates to **True** if and only if **x** is simultaneously greater than zero and less than ten.

Python also natively supports chained comparison expressions, matching standard mathematical notation:

```
0 < x < 10
```

This chained expression functions exactly like the combined logical statement above. When evaluated within numeric contexts or arithmetic equations, Boolean states are treated as numbers, where **True** maps to 1 and **False** maps to 0.

2.3.6 Functions

A function is a reusable block of structured code designed to perform a specific computational task. Python includes an extensive collection of globally accessible, pre-configured tools known as “built-in” functions. In addition to these built-in utilities, developers can define custom functions to handle specialized tasks.

For example, consider the built-in mathematical function **pow()**, which duplicates the behavior of the native exponentiation operator (******):

```
>>> pow(-1, 0.5)
(6.123233995736766e-17+1j)
```

We pass parameters (arguments) into the function call (in this instance, the arguments are `-1` and `0.5`); the function executes its internal instructions and passes a computed result back to the caller. Individual functions accept varying numbers or types of arguments and return different data types depending on their purpose.

2.3.7 Order of Operations

When an expression contains multiple operators, Python determines the execution order using established operator precedence rules. For standard mathematical operators, Python follows conventional algebraic rules. The common mnemonic acronym **PEMDAS** (**P**arentheses, **E**xponentiation, **M**ultiplication/**D**ivision, **A**ddition/**S**ubtraction) summarizes these precedence levels:

- **Parentheses** possess the highest priority. They can enforce a specific execution order regardless of standard precedence rules, or clarify complex equations to improve code readability (e.g., `(minute * 100) / 60`).
- **Exponentiation** has the next highest priority level. Consequently, an expression like `2**1+1` evaluates to 3 (not 4), and `3*1**3` evaluates to 3 (not 27).
- **Multiplication and Division** share identical precedence levels, which are lower than exponentiation but higher than **Addition and Subtraction**.
- Operators that share the exact same precedence level are evaluated sequentially from left to right.

When writing complex equations, we highly recommend using explicit grouping parentheses to ensure that operations execute in the intended order and to avoid subtle logic bugs.

Bitwise Operators

Bitwise operators manipulate integer values at the level of individual raw bits. Instead of treating an integer as a single numeric scalar value, these operators treat the value as a sequential string of bits encoded in two's-complement binary format.

Compound Assignment Operators

In addition to the standard assignment operator (`=`), Python provides compound assignment shortcuts that combine an arithmetic or bitwise operation with an assignment step, making the code more concise.

Table 2.2: Operator Precedence Hierarchy in Python Expressions

Operator Group	Operational Meaning	Precedence Hierarchy Level
**	Exponentiation scaling	power Highest priority
*, /, //, %	Multiplicative, floor division, modulo calculations	division, modulo
+, -	Basic arithmetic and subtraction	addition
==, !=	Value equality and inequality evaluations	
>, <, >=, <=, is, in	Relational comparison and collection inclusion testing	
not, and, or	Logical conjunctions and operators	Boolean Lowest priority

These compound assignments are single, atomic lexical tokens; therefore, no whitespaces or characters are permitted between the operator symbol and the equals sign.

Advanced Membership and Identity Operators

Python includes two specialized operators: **is** and **in**. The identity operator (**is**) tests whether two variables point to the exact same underlying object in memory. In contrast, the comparison operator (**==**) evaluates whether the values stored inside those two objects are equal:

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a == b
True
>>> a is b
False
>>> c = a
>>> a is c
True
```

Table 2.3: Bitwise Operators and Expressions

Operator	Functional Meaning	Syntax	Example and Practical Use Case
<code>&</code>	Bitwise AND operation	<code>x & 1</code>	(Isolates the Least Significant Bit)
<code> </code>	Bitwise OR operation	<code>x 1</code>	(Forces the LSB to state 1)
<code>^</code>	Bitwise Exclusive OR (XOR)	<code>x ^ 0xf</code>	(Inverts the final 4 bits of the variable)
<code>~</code>	Bitwise Unary Complement (NOT)	<code>y = ~x</code>	(Inverts all bits of the target integer)
<code><<</code>	Left logical bit shift	<code>x << 2</code>	(Shifts bits left twice, padding vacant LSB positions with zeros)
<code>>></code>	Right arithmetic bit shift	<code>x >> 2</code>	(Shifts bits right twice, preserving the original sign bit)

```
>>> a[0] = 9
>>> print(c)
[9, 2, 3]
```

In this snippet, `a` and `b` contain identical values, so `a == b` evaluates to `True`. However, they reference separate, independently allocated list objects in memory, which is why `a is b` returns `False`. Because we assigned `c = a`, variable `c` points directly to the same object address as `a`; thus, modifying a value through `a` dynamically alters the data seen by `c`. The negative counterpart of the identity operator is `is not`.

The membership operator (`in`) tests whether a collection sequence (such as a list, tuple, dictionary, or string) contains a specific target value:

```
>>> s = 'Hello'
>>> 'e' in s
True
```

The negative counterpart of the membership operator is `not in`.

Table 2.4: Compound Assignment Statements

Operator	Shortcut	Explicit Breakdown	Structural	Equivalent Expression	Verbose
+=		a += b		a = a + b	
-=		a -= b		a = a - b	
*=		a *= b		a = a * b	
/=		a /= b		a = a / b	
%=		a %= b		a = a % b	
//=		a //= b		a = a // b	
**=		a **= b		a = a ** b	
&=		a &= b		a = a & b	
=		a = b		a = a b	
^=		a ^= b		a = a ^ b	
<<=		a <<= b		a = a << b	
>>=		a >>= b		a = a >> b	

2.4 Data Structures

2.4.1 Lists

A list is a mutable, ordered sequence of elements. The items stored inside a list can be of any data type, and individual lists can hold elements of mixed data types simultaneously. Elements are accessed using a zero-based index enclosed in square brackets (`[]`):

```
>>> lst = [-2, 'alpha', 3.14, [1, 2, 3]]
>>> print(lst[0] + lst[2])
1.14
>>> print(lst[3][2])
3
```

Python also supports negative indexing, which accesses items relative to the end of the list. An index of `-1` points to the absolute final item, `-2` points to the second-to-last item, and so forth:

```
>>> print(lst[-1])
[1, 2, 3]
```

To extract a sub-range of elements, you slice the list using the colon (`:`) operator. Slicing syntax accepts up to three parameters: `L[start:stop:step]`. This returns a fresh sub-list containing elements from the `start` index up to, but not including, the `stop` index, moving at the specified increment (`step`). If omitted, `start` defaults to the beginning of the list, `stop` defaults to the end of the list, and `step` defaults to 1:

```
>>> a = list(range(10)) # Generates a mutable list: [0, 1, ..., 9]
>>> b = a[:]             # Generates a shallow copy of the entire list
>>> c = a[::-1]          # Reverses the entire list order
>>> print(c[::2])        # Extracts every second element
[9, 7, 5, 3, 1]
```

Remember that the `start` boundary index is inclusive, whereas the `stop` boundary index is strictly exclusive.

The following script demonstrates several common built-in methods for list manipulation:

```
#!/usr/bin/python3
# -*- coding: utf-8 -*-

season = ['立春', '立夏 ', '立秋', '立冬']
season.insert(1, '雨水')           # Inserts element at index 1
season.extend(('小雪', '大雪'))    # Appends multiple elements
target_idx = season.index('立秋')  # Locates index of target string
deleted_item = season.pop(target_idx) # Deletes and returns item at index
season[-1] = '大寒'                # Modifies element at final index
season.sort()                      # Sorts items alphanumerically
print('清明' in season)            # Evaluates list membership
```

Calling the `.copy()` method on a list generates a new top-level list container object. However, if any element inside that list is a mutable object (such as a nested sub-list), Python copies references to those nested objects rather than duplicating them. This behavior is known as a **shallow copy**:

```
>>> lst1 = [['中秋', '重阳'], 8.15, 9.9]
>>> lst2 = lst1.copy()
>>> lst2[0][1] = '清明'
>>> lst2[1] = 4.5
>>> print(lst1)
[['中秋', '清明'], 8.15, 4.5]
```

2.4.2 Tuples

A tuple is an ordered sequence of values that behaves similarly to a list. It can store elements of any data type, and items are accessed using standard integer indexing. However, the critical architectural difference is that tuples are completely **immutable**. Once a tuple is instantiated, you cannot modify its values, append new elements, or shrink its size.

Syntactically, a tuple is defined as a comma-separated sequence of values; however, wrapping tuples in parentheses `()` is recommended to clearly distinguish them from other structures:

```
>>> t = ('a', 'b', 'c', 'd', 'e')
>>> t = 'a', 'b', 'c', 'd', 'e'
```

To instantiate a tuple containing only a single element, you must include a trailing comma; otherwise, the parser will treat the expression as a standard string or integer enclosed in grouping parentheses:

```
>>> t1 = ('a')
>>> type(t1)
<class 'str'>
>>> t2 = ('a',)
>>> type(t2)
<class 'tuple'>
```

2.4.3 Sets

A Python set is an unordered collection of unique elements. Sets do not permit duplicate entries. They are commonly used to filter duplicate data out of sequences or to perform mathematical set operations such as unions, intersections, differences, and symmetric differences.

The primary architectural properties of sets include:

- **Unordered Structure:** Elements are not maintained in any specific sequence or index arrangement. Because sets are unordered, they do not support indexing or slicing; individual items cannot be accessed using an integer index key.
- **Guaranteed Uniqueness:** Duplicate entries are automatically dropped when adding items to a set.

- **Mutable Containers:** While the elements inside a set must be immutable types, the set container itself is mutable. It can dynamically grow or shrink as items are added or removed.

2.4.4 Dictionaries

Dictionaries represent Python’s optimization of associative arrays, mapping tables, or hash maps. A dictionary manages a collection of **key:value** pairs, where each unique key maps directly to a corresponding data value. Keys must be immutable data types (such as strings, integers, or tuples):

The primary architectural properties of dictionaries include:

- **Unordered Association:** Items are accessed by their lookup keys rather than by an integer position or insertion sequence.
- **Mutable Interface:** Dictionaries can be altered in place, and they can expand or contract dynamically as pairs are modified.

```
zhang3 = {}                                # Instantiates an empty dictionary
                                           #      (equivalent to dict())
zhang3['name'] = 'Zhang3'                  # Inserts a key-value pair
                                           #      ('name' maps to 'Zhang3')
li4 = zhang3.copy()                        # Generates a shallow copy object
li4['age'] = 23                             # Appends a new item into the copied
                                           #      dictionary
print(zhang3, li4)
```

2.5 Python Program Architecture

2.5.1 Program Structure

Consider the following standalone script that determines whether an integer is a prime number:

Listing 2.2: prime.py, Script to Verify Prime Numbers

```
1 #!/usr/bin/python3
2 # -*- coding: utf-8 -*-
3
```



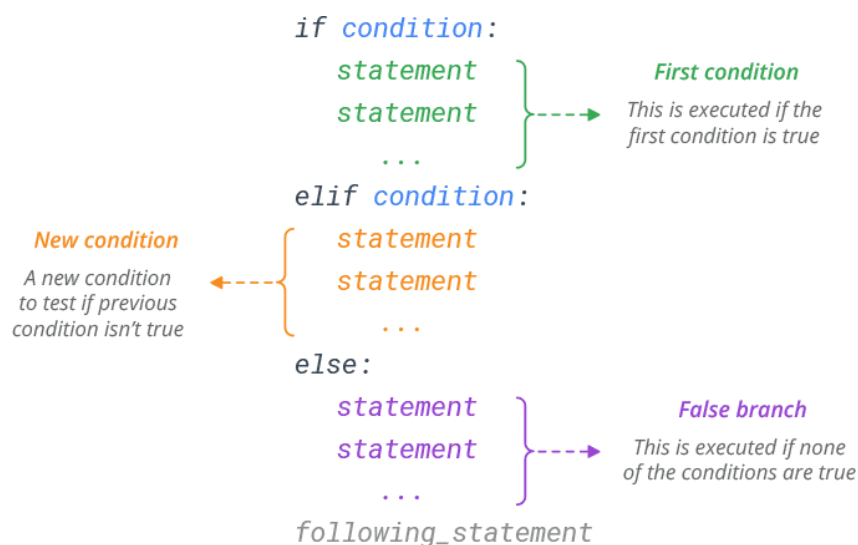
```
4 x = 127
5
6 def is_prime(val):
7     for i in range(2, val):
8         if val % i == 0:
9             return False
10    return True
11
12 if is_prime(x) is True:
13     print(x, 'is a prime.')
14 else:
15     print(x, 'is not a prime.')
```

The Python interpreter processes script files sequentially, line-by-line, starting from the first executable top-level statement. Function and class declarations can be placed anywhere throughout this execution flow. The **def** keyword defines a named function block, and all internal statements belonging to that function must be indented uniformly.

To maintain clean code formatting, avoid mixing tabs and raw spaces within the same execution block, as different text editors interpret tab widths inconsistently. The official Python style guide strongly recommends using four spaces per indentation level.

2.5.2 Conditional Execution

To implement meaningful program logic, scripts must evaluate conditions and alter execution flows accordingly. Python handles conditional execution using the keywords **if**, **elif** (else if), and **else**.



- **Basic Conditional Branching:**

```
a = '1234'
if isinstance(a, int):
    print('a is an integer.')
```

The Boolean expression evaluated immediately following the **if** keyword is the condition. The condition statement must end with a trailing colon (:), and the code lines that execute if the condition is true must be indented.

- **Alternative Binary Execution Paths:**

```
a = 2018
if a > 0:
    print('a is a positive number.')
else:
    print('a is not a positive number.')
```

Because the evaluated condition must resolve to either true or false, exactly one of the two code blocks will execute.

- **Chained Multiple Branch Conditionals:**

```
if a > 0:
    print('a is a positive number.')
elif a < 0:
    print('a is a negative number.')
else:
    print('a is zero.')
```

The interpreter evaluates each condition sequentially. If a condition returns false, control flows to the next branch. When a condition evaluates to true, its corresponding code block executes and the entire conditional structure terminates. Even if multiple conditions are true, only the first valid branch will execute.

- **Inline Conditional Expressions:** Python supports inline conditional expressions (ternary operations) to assign values or select concise actions based on a condition:

```
status_str = 'positive' if a > 0 else 'non-positive'
```

2.5.3 Loops and Iterations

Computers are routinely deployed to automate repetitive operations. Python provides two primary looping structures to handle iterations: **for...in** loops and **while** loops. Loop execution flows can be controlled using the keywords **continue**, **break**, and **pass**.

The **break** statement terminates the innermost active loop, passing execution control to the first line of code immediately following the loop block. The **continue** statement stops the current loop iteration and immediately jumps execution back to the top of the loop block to evaluate the next item or loop condition. The **pass** statement is a null operation; it performs no action and serves strictly as a structural placeholder when Python syntax requires an indented code block.

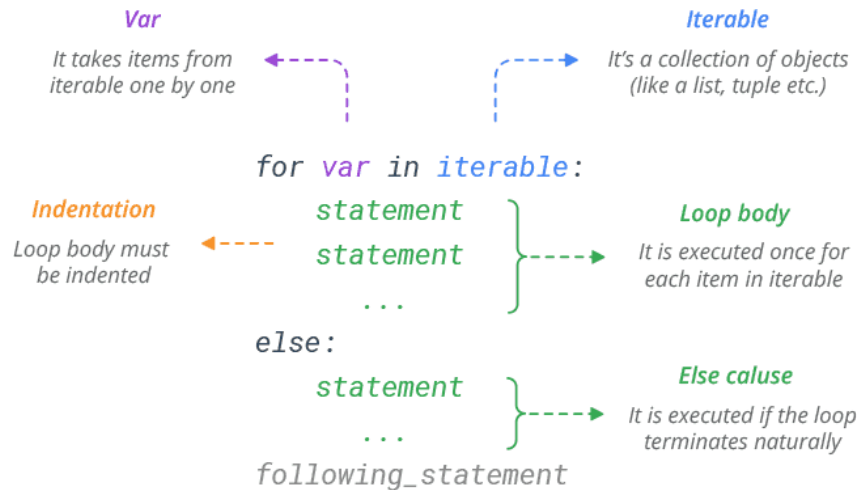
- The **for...in** Collection Loop:

```
days = [('Jan', 31), ('Feb', 28), ('Mar', 31), ('Apr', 30)]
for month, count in days:
    print('There are {} days in {}'.format(count, month))
```

- The Integrated **for...else** Structure:

```
numbers_list = [3, 9, 4, 2, 0, 5]
for x in numbers_list:
    if x <= 0:
        print('Found a non-positive number.')
        break
else:
    print('All numbers in the list are positive.')
```

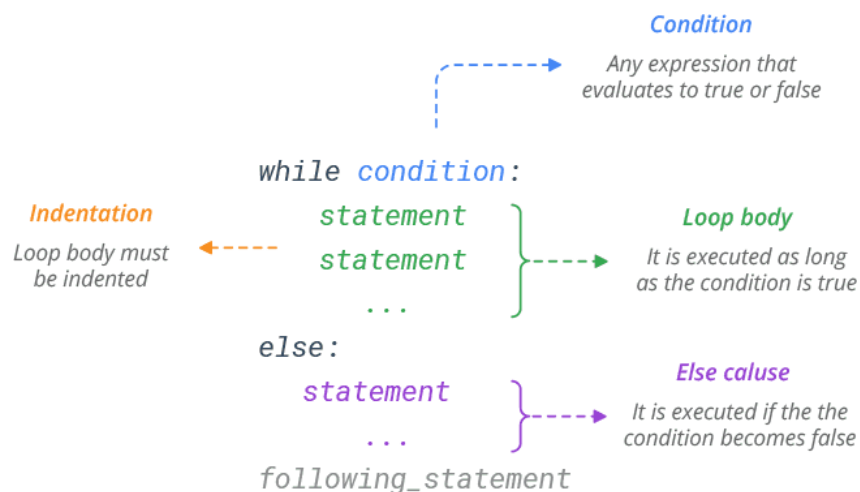
Note that the terminal **else** block belongs to the loop structure itself, rather than the internal **if** statement. The code inside the loop's **else** block executes if and only if the loop runs to natural completion without encountering a **break** statement.



- The Conditional `while` Loop:

```

import time
countdown = 10
while True:
    print(countdown)
    countdown -= 1
    if countdown <= 0:
        break
    time.sleep(1)
print('Time is up!')
  
```



For concise list initialization, looping constructs can be nested directly within bracket declarations to perform **list comprehensions**:

```

# Efficiently populates a list with squared values: [0, 1, 4, ..., 81]
squares_table = [x * x for x in range(10)]
  
```

2.5.4 Functions and Scope

Functions maximize code reuse, minimize logic duplication, and break complex programs into manageable blocks. The **def** statement creates a function object and binds it to an identifier:

```
def function_name(arg1, arg2, argN):  
    # Function body statements
```

Function bodies often include a **return** statement, which can be placed anywhere within the block. Executing a **return** statement immediately exits the function and sends a calculated value back to the caller.

Functions also support default argument values, which make specific parameters optional during invocation. If a caller omits an optional argument, Python assigns it its default value before executing the function:

```
>>> def display_coordinates(x, y, label='Point'): print(x, y, label)  
...  
>>> display_coordinates(1, 2, 'Vertex')  
1 2 Vertex  
>>> display_coordinates(y=3, x=5)  
5 3 Point
```

To accept an arbitrary number of inputs, functions can declare parameters prefixed with single (*) or double (**) asterisks. A single asterisk (***args**) captures excess positional inputs into an immutable tuple, whereas double asterisks (****kwargs**) capture excess keyword arguments into a mutable dictionary:

```
>>> def sum_all(*args): print(sum(args))  
...  
>>> sum_all(1, 3, 5, 7, 9)  
25  
>>> def print_metadata(**kwargs): print(kwargs.keys(), kwargs.values())  
...  
>>> print_metadata(name='Zhang', age=35)  
dict_keys(['name', 'age']) dict_values(['Zhang', 35])
```

Anonymous, single-expression utility functions can be created using the inline **lambda** expression syntax:

```
>>> compute_sum = lambda x, y, z: x + y + z  
>>> compute_sum(2, 3, 4)  
9
```

2.5.5 Modules and Package Management

Python is an extensible language. Complex programs can be organized across separate files called **modules**. Each individual script file operates as an independent module that can export functions, variables, and classes to be used by other files. To access names defined inside an external module, you must first import it using the **import** keyword:

```
import math
print(math.sin(math.pi * 0.5))
```

By default, functions imported via a standard **import module** statement must be prefixed with the module's name to prevent namespace clashes. Alternative import styles let you omit this prefix or rename modules:

```
from math import sin
print(sin(1))           # Executable
print(sin(pi / 2))      # Raises a NameError because 'pi' was
                        # not explicitly imported
...
from math import *
print(cos(pi / 3))      # Executable; imports all symbols
                        # from the module
...
import math as m
print(m.exp(1.0))       # Executable; renames the module namespace
                        # shortcut to 'm'
```

The broader Python developer ecosystem shares thousands of open-source third-party modules via the official PyPI (*Python Package Index*) repository (<https://pypi.org>). Because many packages have cascading dependencies, Python provides an automated command-line tool called **pip** (*Package Installer for Python*) to search, download, compile, and manage these libraries:

```
# pip install numpy      # Resolves dependencies when installing
# pip list               # Lists all installed packages
# pip uninstall numpy    # Safely removes the package
```

2.6 Summary

Python is a powerful, high-level language used extensively across data analysis, machine learning, and hardware automation. This section reviewed the core fundamentals of Python programming. For rapid lookup, Table 2.5 and Table 2.6 compile all reserved system keywords and globally available built-in functions.

Table 2.5: Reserved Keywords in Python Programming

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

You can inspect the reserved keyword list inside an interactive shell by running:

```
>>> import keyword
>>> print(keyword.kwlist)
```

Table 2.6: Globally Available Built-in Interpreter Functions

<code>abs()</code>	<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>
<code>all()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>any()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>ascii()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>bin()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bool()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>breakpoint()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>
<code>compile()</code>	<code>globals()</code>	<code>map()</code>	<code>reversed()</code>	<code>__import__()</code>
<code>complex()</code>	<code>hasattr()</code>	<code>max()</code>	<code>round()</code>	

Chapter 3

I/O Interfaces on Raspberry Pi

3.1 Raspberry Pi

3.1.1 Short Introduction

The Raspberry Pi is a series of small single-board computers developed by a non-profit organization (the Raspberry Pi Foundation) in the United Kingdom in association with Broadcom Inc. The original Raspberry Pi project leaned towards promoting the teaching of basic computer science in schools and developing countries. Due to its low cost, modularity, and open design, it is now widely used across a diverse range of technical application fields. By December 2019, a total of 30 million Raspberry Pi units had been sold globally.¹

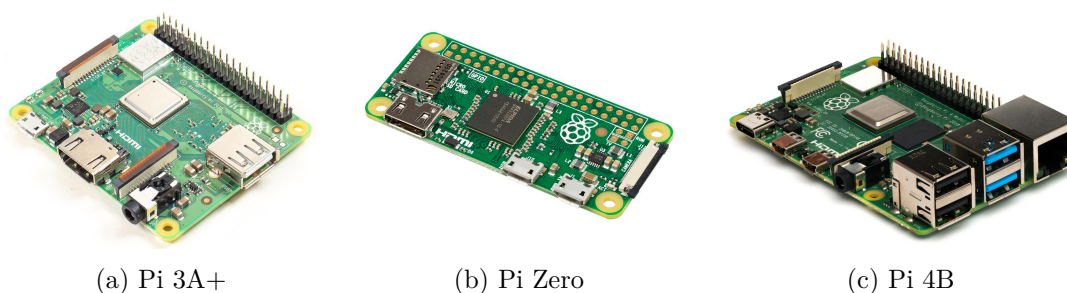


Figure 3.1: Different Models of Raspberry Pi

A single-board computer is a complete computer built on a single circuit board, containing the microprocessor(s), memory, input/output (I/O) facilities, and other fundamental features required of a fully functional computer. Single-board computers

¹Tweet by Eben Upton (CEO of Raspberry Pi Trading), December 14, 2019.

are commonly manufactured as demonstration or evaluation platforms, for educational instruction, or for deployment as embedded hardware controllers.

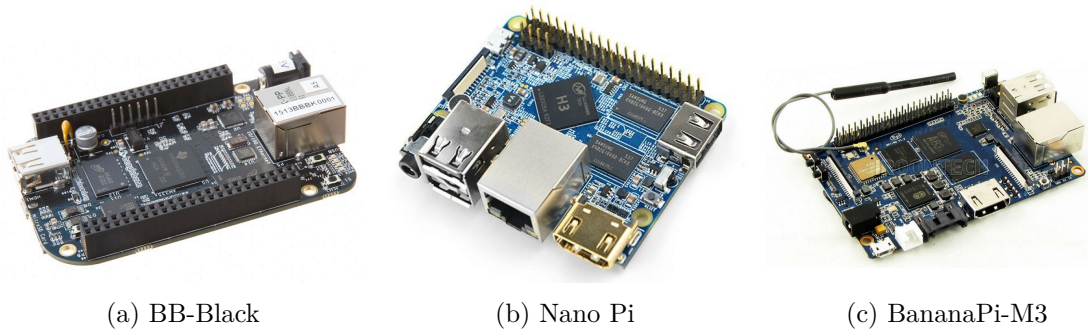


Figure 3.2: Alternative Single-Board Computer Configurations

All modern Raspberry Pi models feature a Broadcom system on a chip (SoC) with an integrated ARM-compatible central processing unit (CPU) and an on-chip graphics processing unit (GPU). The processing speeds across the product line range from 700 MHz to over 1.5 GHz, paired with onboard memory allocations scaling from 256 MB up to 8 GB of RAM. Secure Digital (SD) cards in MicroSD form factors are utilized to host the operating system and provide persistent non-volatile storage.

The boards feature up to four integrated USB ports, composite video outputs, and HDMI ports for digital display connectivity. Low-level hardware interfacing is supported via a row of general-purpose input/output (GPIO) pins, which natively expose standard serial communication protocols such as I²C, SPI, and UART.

3.2 Low-Level Hardware Expansion Interfacing

The primary physical feature distinguishing single-board embedded computers from traditional desktop workstations is the row of hardware pins exposed along the edge of the PCB. This pin array is designated as the general-purpose input/output interface connector. Through these pins, the system can interact directly with peripheral hardware components such as switches, sensors, solid-state relays, display modules, and operational drivers.

The physical layout and pin allocation scheme of the Raspberry Pi hardware connector is illustrated in Figure 3.1. This header contains 40 distinct pins spaced at a standard 2.54mm pitch. The pins are organized into three primary functional groups:

1. **Power Supply Rails:** Direct power outputs connected to the system's power traces. These include two 5V power pins (Pins 2 and 4), two 3.3V power pins

Table 3.1: Physical GPIO Pin Mapping of the Raspberry Pi

GPIO Connector					
BCM GPIO	Name	Header		Name	BCM GPIO
	3.3v	1	2	5v	
GPIO2	SDA1	3	4	5v	
GPIO3	SCL1	5	6	GND	
GPIO4	GCLK	7	8	TxD	GPIO14
	GND	9	10	RxD	GPIO15
GPIO17	GEN0	11	12	GEN1	GPIO18
GPIO27	GEN2	13	14	GND	
GPIO22	GEN3	15	16	GEN4	GPIO23
	3.3v	17	18	GEN5	GPIO24
GPIO10	MOSI	19	20	GND	
GPIO9	MISO	21	22	GEN6	GPIO25
GPIO11	SCLK	23	24	CE0	GPIO8
	GND	25	26	CE1	GPIO7
GPIO0	ID-SD	27	28	ID-SC	GPIO1
GPIO5		29	30	GND	
GPIO6		31	32		GPIO12
GPIO13		33	34	GND	
GPIO19		35	36		GPIO16
GPIO26		37	38		GPIO20
	GND	39	40		GPIO21
BCM GPIO	Name	Header		Name	BCM GPIO

note: GPIO18 and GPIO19 can be used as hardware PWM function.

(Pins 1 and 17), and multiple Ground references (GND). These pins supply physical voltage to drive low-current external hardware peripherals.

2. **General-Purpose I/O (GPIO) Pins:** Software-configurable pins that can be independently toggled via user applications to serve as digital inputs or digital outputs.
3. **Alternative Protocol Pins:** Multi-functional pins that can be multiplexed internally to handle hardware-driven serial communication protocols, including I²C,

SPI, and UART interfaces.

3.3 I/O System Usages

A GPIO is an uncommitted digital signal pin on an integrated circuit or electronic circuit board whose behavior – including whether it acts as input or output – is controllable by the user at run time. To access these resources in Linux user space, let's change our working directory to `/sys/class/gpio`:

```
# cd /sys/class/gpio
# ls -l
```

We find following files and directories:

```
/sys/class/gpio
|-- export      (to create a GPIO node)
|-- unexport    (to delete a GPIO node)
`-- gpiochip0   (symbolic link)
```

The file `export` and `unexport` are used to create or delete GPIO nodes. With these nodes we can read or write GPIO levels. To access a pin (GPIO18 or pin 12 for instance), we need to create a node in user space related with it. With this node, we can read or change GPIO levels.

To create a node is to write a digital number to the file `export` by using redirection technique:

```
# echo 18 > export
```

Now a new symbolic link file (actually it is a directory) is created. Let's enter this directory and list files again:

```
# cd gpio18
# ls
active_low  direction  power      uevent
device      edge       subsystem  value
```

The file `direction` is used to set GPIO mode: “in” for input, “out” for output. The file `value` is used to control or read pin level. When GPIO is set to input, the file `value` is for reading; when GPIO is set to output, writing 0 or 1 to file `value` sets the pin level low or high:

```
# echo "in" > direction
# more value                # print pin level ("0" or "1")
# echo "out" > direction
```

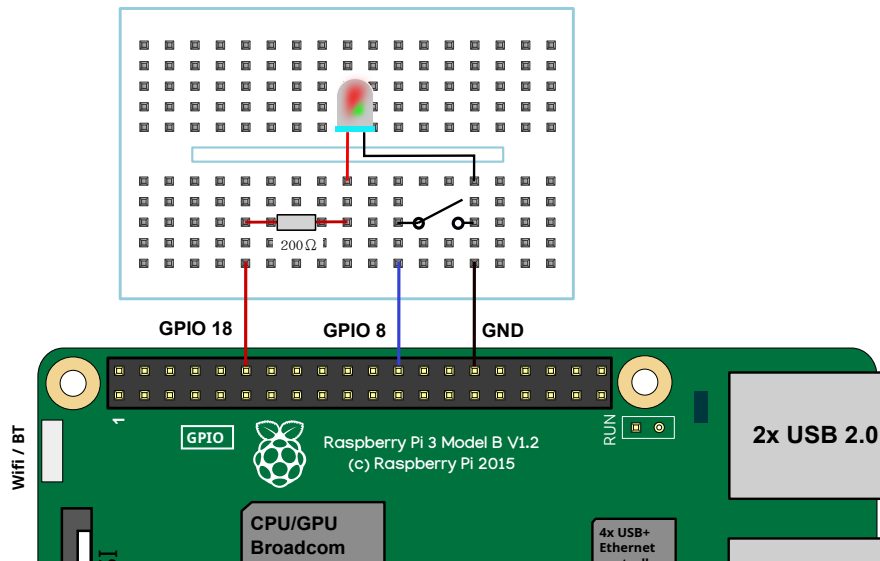


Figure 3.3: Connecting an LED and a Switch to GPIOs

```
# echo 1 > value          # output HIGH
# echo 0 > value          # Output LOW
```

When an LED (Light-Emitting Diode) is connected to this GPIO, you may see it on and off by writing “1” or “0” to file `value`.

There is PWM modules under the directory `/sys/class/pwm/pwmchip0`. To generate a PWM output, one can write a 0 or 1 to create a PWM node (just like GPIO node), enter the subdirectory, set duty cycle and period to the appropriate file.

When nodes are no longer to be used, it's better to remove the node directories from filesystem. The way to remove a node directory is to write a appropriate digit to the file `unexport`, which was written to `export` before.

The on-board LEDs are also controlled through GPIO. On Raspberry Pi 3, Active indicator is connected to GPIO47 and power status indicator is connected GPIO35). Users may enter directory `/sys/class/leds` to access them.

3.4 GPIO Modules for Python

3.4.1 Install a GPIO Module

There are several Python modules to manipulate GPIOs of Raspberry Pi. A flexible module which is written by Ben Croston is preferable and is also suggested by Raspberry community. A simplest way to install this module is using pip on Raspberry Pi:

```
# pip install RPi.GPIO
```

The official released module in PYPI is lack of hardware PWM support (till version 0.7). A patched version, which is hosted on github, can be downloaded and installed by following steps:

1. Get source code from github:

```
# git clone https://github.com/yfang1644/RPi.GPIO
```

2. Enter the source code directory to compile and install:

```
# cd RPi.GPIO
# python3 setup.py build
# python3 setup.py install
```

3. Now the module `RPi.GPIO` is available. Try following operations in Python shell to ensure the module is installed correctly. Note that the module name is **case-sensitive**.

```
# import RPi.GPIO
```

After installation, source code is no longer needed and therefore can be deleted.

3.4.2 Pin Numbering

There are two numbering system the IO pins on a Raspberry Pi within `RPi.GPIO` (see Table 3.1). The first is using the BOARD numbering system. This refers to the pin numbers on the connector of the Raspberry Pi board. The advantage of using this numbering system is that your hardware will always work, regardless of the board revision of the RPi. You will not need to rewire your connector or change your code.

The second numbering system is the BCM numbers. This is a lower level way of working – it refers to the channel numbers on the Broadcom SoC. You have to always work with a diagram of which channel number goes to which pin on the RPi board. Your script could break between revisions of Raspberry Pi boards.

To specify which you are using, call the function `setmode()`:

```
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BOARD)
# or
GPIO.setmode(GPIO.BCM)
```

To detect which pin numbering system has been set (for example, by another Python module):

```
mode = GPIO.getmode()
```

The mode will be GPIO.BOARD, GPIO.BCM. If the pin is not configured, the function returns None.

3.4.3 Set up Pins

The function of a pin must be set up before use – either as an input or as an output. To do so, you need to call function `setup()`:

```
GPIO.setup(channel, GPIO.IN)
# or
GPIO.setup(channel, GPIO.OUT)
```

where `channel` is the pin number or GPIO number based on the numbering system you have specified (BOARD or BCM).

You can also specify an initial value for your output channel, or pull-up/pull-down resistor for your input channel

```
GPIO.setup(channel1, GPIO.OUT, initial=GPIO.HIGH)
GPIO.setup(channel2, GPIO.IN, pull_up_down=GPIO.PUD_UP)
```

Multiple channels can be set up per call, when iterable object is used:

```
chan_list = [11,12]      # add as many channels as you want!
                        # you can tuples instead i.e.:
                        #   chan_list = (11,12)
GPIO.setup(chan_list, GPIO.OUT)
# or
GPIO.setup(chan_list, [GPIO.IN, GPIO.HIGH])
```

Similarly, you can set multiple pin levels by calling function `output()` once, when the pins are configured as output:

```
chan_list = [11,12]      # also works with tuples
GPIO.output(chan_list, GPIO.LOW)
GPIO.output(chan_list, (GPIO.HIGH, GPIO.LOW))
```

At the end of the program, it is good practice to clean up any resources you might have used. After `cleanup()`, all channels you have used back to inputs with no pull up/down. This will avoid accidental damage to Raspberry Pi by shorting out the pins.

```
GPIO.cleanup(channel)      # individual clean up
GPIO.cleanup([channel1, channel2]) # multiple clean up
GPIO.cleanup()             # clean up all pins
```

3.4.4 Use GPIO as Input

Polling

GPIO input reflects the level the pin connects to. you can use function `input()` to read the value of a GPIO pin (polling):

```
val = GPIO.input(channel)
if val == GPIO.LOW:
    ...
elif val == GPIO.HIGH:
    ...
```

Function `input()` returns either 0/GPIO.LOW/False or 1/GPIO.HIGH/True.

To wait for a button press by polling in a loop:

```
while GPIO.input(channel) == GPIO.LOW:
    time.sleep(0.01)
```

This assumes that pressing the button changes the input from LOW to HIGH. CPU reads the pin status every 10 ms.

Interrupts and Edge detection

As we can see from polling, CPU runs busily on reading the status of a pin, and may miss some state changes (when the duration of a button press is shorter than 10ms).

GPIO module can detect the edge. An edge is the change in state of an electrical signal from LOW to HIGH (rising edge) or from HIGH to LOW (falling edge). Quite often, we are more concerned by a change in state of an input than its value. This change in state is an event.

To avoid missing a button press while your program is busy doing something else, there are several ways to get round this:

- the `wait_for_edge()` function
- the `event_detected()` function
- a threaded callback function that is run when an edge is detected

The `wait_for_edge()` function is designed to **block** execution of your program until an edge is detected. In other words, the example above that waits for a button press could be rewritten as:


```
chn = GPIO.wait_for_edge(channel, GPIO.RISING, timeout=10)
if chn is None:
    print('Timeout occurred')
else:
    print('Edge detected on channel', channel)
```

You can set `RISING`, `FALLING` or `BOTH` event to be detected. The advantage of doing it this way is that it uses a negligible amount of CPU, so there is plenty left for other tasks.

The `event_detected()` function is designed to be used in a loop with other things, but unlike polling, it is not going to miss the change in state of an input while the CPU is busy working on other things. This could be useful when using something like GUI programs where there is a main loop listening and responding to GUI events in a timely basis.

```
GPIO.add_event_detect(channel, GPIO.RISING)
do_something()
if GPIO.event_detected(channel):
    print('Button pressed')
```

Note that you can detect events for `RISING`, `FALLING` or `BOTH`.

Callback Functions

RPi.GPIO runs a second thread for callback functions. This means that callback functions can be run at the same time as your main program, in immediate response to an edge. For example:

```
def my_callback(channel):
    print('This is a edge event callback function!')
    print('Edge detected on channel %s' % channel)
    print('The callback runs in a different thread to main program')

GPIO.add_event_detect(channel,
                      GPIO.RISING,
                      callback=my_callback)
...the rest of your program...
```

Callback functions are special functions. They are not called explicitly, since the events are unpredictable. The arguments of a callback function is determined by the designer, not user. And more often, callback function has no return value, because no caller can get the return value.

If you want more than one callback function for an event:

```
def my_callback_one(channel):
    print('Callback one')

def my_callback_two(channel):
    print('Callback two')

GPIO.add_event_detect(channel, GPIO.RISING)
GPIO.add_event_callback(channel, my_callback_one)
GPIO.add_event_callback(channel, my_callback_two)
```

Note that in this case, the callback functions are run sequentially, not concurrently. This is because there is only one thread used for callbacks, in which every callback is run, in the order in which they have been defined. Therefore, it's possible that one channel has multiple callbacks, but **multiple channels cannot share a common callback function**.

Switch Debounce

You may notice that the callbacks are called more than once for each button press. This is as a result of what is known as “switch bounce”. There are two ways of dealing with switch bounce: hardware approach and software approach. Add a capacitor across the switch to filter the high frequency signal may improve debouncing. To debounce using software, add the `bouncetime` parameter in milliseconds to a function where you specify a callback function:

```
# add rising edge detection on a channel,
# ignoring further edges for 200ms for switch bounce handling
GPIO.add_event_detect(channel,
                       GPIO.RISING,
                       callback=my_callback,
                       bouncetime=200)

# or
GPIO.add_event_callback(channel,
                        my_callback,
                        bouncetime=200)
```

If for some reason, your program no longer wishes to detect edge events, it is possible to stop them:

```
GPIO.remove_event_detect(channel)
```

3.4.5 Ultrasonic Ranging

Ultrasonic ranging entails transmitting a sound wave and measuring the time that it takes for the sound wave to reflect off of an object and back to the transducer. The reflection time is proportional to the distance between the object and the transducer. In this implementation, a blanking interval is required between signal transmission and reception.

The ultrasonic ranging module SR04 provides stable readings and accurately measures distance. This module can be used for a number of projects such as fluid level in a tank, distance to obstacles, and other sensing tasks.

The processor initiates IO trigger by supplying at least $10\mu\text{s}$ high level pulse to “trig” pin. The module will automatically send 8 40kHz square waves, drive “echo” pin to high, and detect the returning pulse signal. When returning signal is received, the device pulls the “echo” back to low. The processor should mark the time that the level changes of the “echo” pin (Figure 3.4).

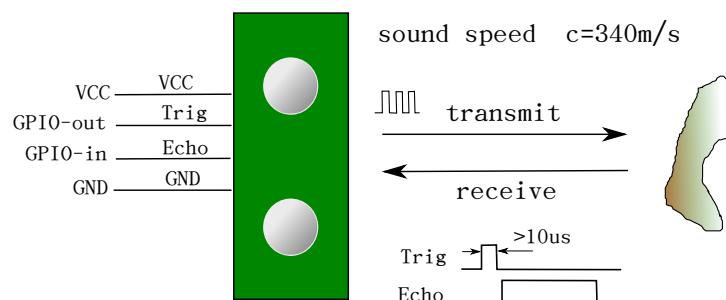


Figure 3.4: Ultrasonic Ranging

A test code for ultrasonic ranging is shown in List 3.1.

Listing 3.1: ranging.py, Ranging Using a Ultrasonic Module

```
TRIG, ECHO = ....

def ranging():
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.000012)
    GPIO.output(TRIG, GPIO.LOW)
    while GPIO.input(ECHO) == GPIO.LOW:
        pass
```

```

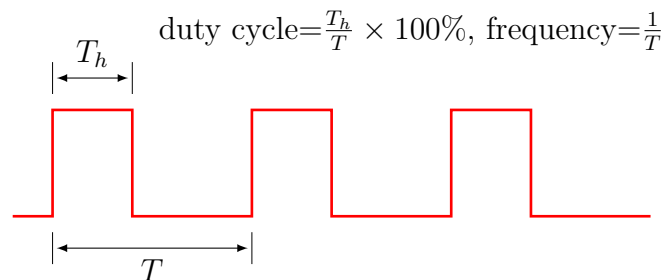
t1 = time.time()                # mark the rising edge
while GPIO.input(ECHO) == GPIO.HIGH:
    pass
t2 = time.time()                # mark the falling egde
return (t2 - t1 )*34000/2       # distance in cm

```

3.5 Using PWM

3.5.1 What is a PWM

Pulse-width modulation (PWM), or pulse-duration modulation (PDM), is a modulation technique used to encode a message into a pulsing signal.



The term **duty cycle** describes the proportion of “on” time to the regular interval or “period” of time. Duty cycle is expressed in percent, 100% being fully on. A lower duty cycle corresponds to lower power.

3.5.2 Accessing PWM in Userspace

Under the directory of `/sys/class/pwm`, several subdirectory for pwmchips are listed, one of them looks like:

```

/sys/class/pwm/pwmchip0
| ...
|-- export          (to create PWM node)
|-- unexport        (to delete PWM node)
`-- npwm            (number of PWMs)

```

The file `npwm` describes how many pins this group has. Each group has maximum 2 PWM pins valid. we can write an index “0” or “1” to file `export` to create PWM node. Under each node directory, there are files for the user to set PWM property:

```

/sys/class/pwm/pwmchip0
| ...

```

```

|-- export          (to create PWM node)
|-- unexport        (to delete PWM node)
|-- npwm            (number of PWMs)
`-- pwm0
    | ...
    |-- enable      (0 or 1)
    |-- capture
    |-- period      (time in ns)
    |-- duty_cycle  (time in ns)
    `-- polarity    (normal or inversed)

```

Writing an integer number to file `period` will set the output waveform to a certain period in nano seconds. Writing an integer number to `duty_cycle` sets its duty cycle periods in nano seconds. The duty cycle must be less or equal to period.

3.5.3 PWM Controlling using RPi.GPIO

Python's PWM module within RPi.GPIO enables to access the PWM on board, to easily and accurately generate a PWM output signal of a given duty cycle and frequency.

To create a PWM instance:

```

GPIO.setup(channel, GPIO.OUT)
p = GPIO.PWM(channel, frequency)

```

A pin used as PWM must be set as output. Function `start()` starts PWM output:

```

p.start(dc)          # duty cycle (0.0 <= dc <= 100.0)

```

Once the PWM instance variable `p` is created, it's not suitable to create it again before removing it (`cleanup()`). Function `ChangeFrequency()` and `ChangeDutyCycle()` are used to change the output of the PWM:

```

p.ChangeFrequency(freq)  # set new frequency in Hz
p.ChangeDutyCycle(dc)    # 0.0 <= dc <= 100.0

```

Function `stop()` stops the PWM output, until next `start()` function call.

```

p.stop()

```

Note that PWM will also stop if the instance variable `p` goes out of scope.

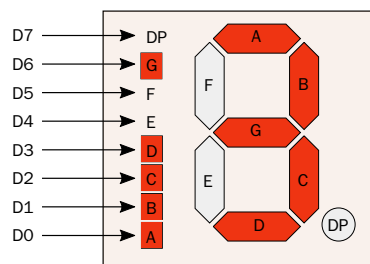
Since software PWMs in RPi.GPIO module are programmed as thread, all GPIO pins can be configured as PWM. But only GPIO18 and GPIO19 are hardware PWM pins. Hardware PWM's clock are generated by crystal oscillators, which is far more accurate than thread in timeslice and required almost no CPU instructions.

3.6 I2C Module

3.6.1 Single Segments Display

A 7-segment display (also called 8-segment display, with a decimal point) is a form of electronic display device for displaying decimal numerals. Such displays are widely used in digital clocks, electronic meters, basic calculators, and other electronic devices that display numerical information.

The display is composed of 8 LEDs, with all of the cathodes (negative terminals) or anodes (positive terminals) connected together and brought out to a common pin; this is referred to as a “common cathode” or “common anode” device. When D7~D0 connect to data bus of a computer, if you want to display “3”, you can write a “0x4F” to the common cathode device, or “0xB0” to the common anode one (Figure 3.5a).



(a) 7-Segment Display



(b) 4 Displays in a Module

Figure 3.5: 7-Segment LED Display

3.6.2 Multiple Display

Each 7-segment display needs at least 9 pins to control. With this kind of structure, multiple displays will not be practical to use.

A multiple segment LED display with 4 digitals is shown in Figure 3.5b. In order to send multiple digital signal with less pins, serial communication techniques are introduced. This module is an I²C (Inter-Integrated Circuit, or I2C, IIC, pronounced “I square C” or “I two C”) communication peripheral.

The I²C bus is a synchronous, multi-master, multi-slave, packet-switched serial communication bus (See Figure 3.6). It is classified as a two-wire serial interface because it requires only two physical signal wires to transmit and receive data between devices:

- **SDA (Serial Data):** The bidirectional line used to transmit data bits between masters and slaves.

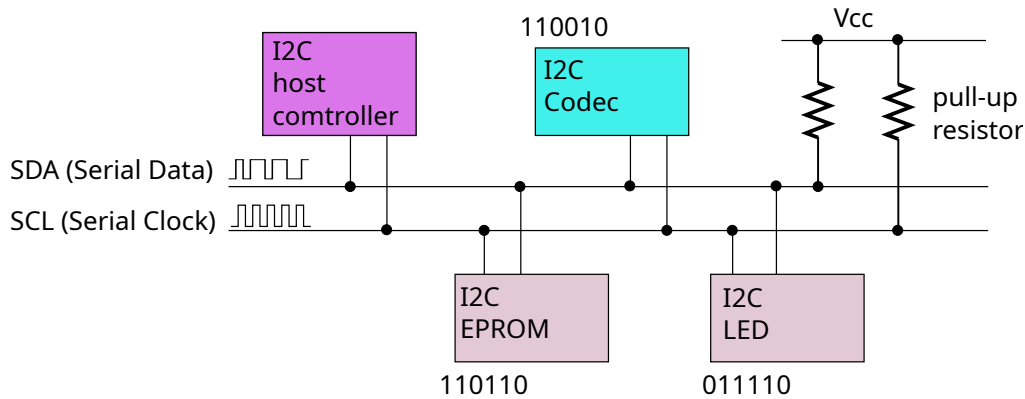


Figure 3.6: Generalized I2C Connection Diagram

- **SCL (Serial Clock):** The clock signal line generated by the active master device to synchronize data sampling along the bus.

The foundational timing characteristics and signaling constraints of the I²C protocol are illustrated in Figure 3.7.

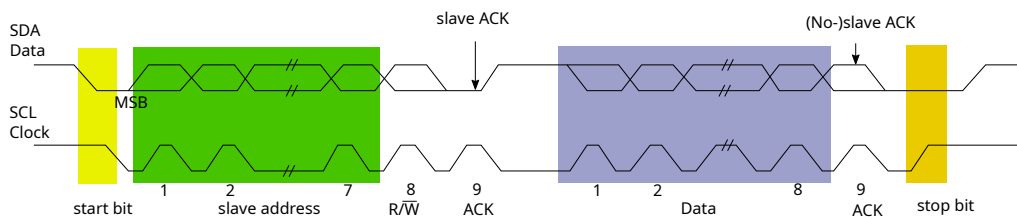


Figure 3.7: Standard I2C Bus Signaling and Timing Constraints

Depending on your distribution's default kernel configuration, you may occasionally need to manually insert the underlying hardware driver modules into the kernel space:

```
# modprobe i2c_dev
# modprobe i2c_bcm2835
```

The physical I²C bus interface exposed on the Raspberry Pi's external connector maps internally to the system device node `i2c-1`. On the physical header, Pin 3 (GPIO2) serves as the serial data line (SDA), and Pin 5 (GPIO3) serves as the serial clock line (SCL). You can verify the presence and permissions of the corresponding device driver file within the system device directory:

```
# ls -l /dev/i2c-1
crw----- 1 root  root  89,   1 Jan 24  2021 /dev/i2c-1
```

The `i2cdetect` program will probe all the addresses on a bus, and report whether any devices are present.

```
# i2cdetect -y 1
      0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- 24 25 26 27 -- -- -- -- -- -- -- --
30: -- -- -- -- 34 35 36 37 -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

In this setup, each connected 7-segment LED display module is configured with a unique hardware slave address on the I²C bus (typically mapping to addresses 0x34, 0x35, 0x36, and 0x37, respectively). By executing targeted block write operations to a specific slave address, user applications can selectively illuminate the display’s segments to output numbers or characters.

The option “-y” will disable interactive mode, the “1” indicates that we are scanning for I2C devices on I2C bus 1 (e.g. /dev/i2c-1).

You may write a byte 0x3f to I2C register 0x34 with `i2cset`:

```
# i2cset -y 1 0x34 0x3f
```

`i2cset`, `i2cget`, etc. are useful I2C commands that manipulate I2C devices. Examples:

- Write the value 0x5000 to a 16-bit register 0x20 to a device on at 7-bit I2C address 0x68 on bus 1 with no user confirmation:

```
# i2cset -y 1 0x68 0x20 0x5000 w
```

Warning: `i2cset` can be dangerous if used improperly. It can confuse your I2C bus, cause data loss, or have more serious side effects.

- Read the content of an I2C device at 7-bit address 0x68 on bus 1 from an 8-bit register at 0x20 after user confirmation:

```
# i2cget 1 0x68 0x20
```

Warning: `i2cget` can be dangerous if used improperly. Some chips can process a I2C read command as a write command and write incorrect data to a register.

- Dump the content of an I2C device at 7-bit address 0x68 on bus 1 using the default read byte mode with a user confirmation:


```
# i2cdump 1 0x68
```

- Dump the content of an I2C device at 7-bit address 0x68 on bus 1 using I2C block read transactions without user confirmation:

```
# i2cdump -y 1 0x68 i
```

Warning: `i2cdump` can be dangerous if used improperly. Most notably, the `c` mode starts with WRITING a byte to the chip. **Do not use `i2cdump` on random addresses.**

- On bus 0, from an EEPROM at address 0x50, read 8 bytes from offset 0x64 (first message writes one byte to set the memory pointer to 0x64, second message reads from the same chip):

```
# i2ctransfer 0 w1@0x50 0x64 r8
```

The description blocks are composed as `{r|wn}[address]`. `{r|w}` specifies if the message is read or write, `n` specifies the number of bytes read or written in this message. It is parsed as an unsigned 16-bit integer. `[address]` specifies the 7-bit address of the chip to be accessed for this message, and is an integer.

Warning: `i2ctransfer` can be dangerous if used improperly. Some chips can process a I2C read command as a write command and write incorrect data to a register.

One of the premier Python modules available for I2C to interact with this bus is `smbus2`. You can install this module on your system using the `pip` package manager:

```
# pip install smbus2
```

With `smbus2`, you can easily handle I2C devices as following:

```
from smbus2 import SMBus, i2c_msg
# Open a handle to "/dev/i2c-1", representing the I2C bus.
bus = SMBus(1)
# or
bus = SMBus();
bus.open(1)
# or
bus.open('/dev/i2c-1')

# Write a single byte to a device (addr)
```

```

bus.write_byte(addr, byte, force=None)

# Write a byte to a specified register
bus.write_byte_data(addr, register, value, force=None)

# Write a block of byte data to a specified register.
bus.write_block_data(addr, register, data, force=None)

# Write a block of byte data to a specified register.
bus.write_i2c_block_data(addr, register, data, force=None)

# Read a single byte from a device (addr)
byte = bus.read_byte(addr, force=None)

# Read a block of up to 32-bytes from a specified register.
data = bus.read_block_data(addr, register, force=None)

# Read a single byte from a designated register.
data = bus.read_byte_data(addr, register, force=None)

# Read a block of byte data from a specified register.
data = bus.read_i2c_block_data(addr, register, length, force=None)

'''
i2c_rdwr(*i2c_msgs) Combines a series of i2c read and write
operations in a single transaction (with repeated start bits
but no-stop bits in between). This method takes i2c_msg
instances as input, which must be created first with
i2c_msg.read() or i2c_msg.write()
'''
command = [0x11, 0x22, 0x33, 0x44, 0x55]
write = i2c_msg.write(address, command)
bus.i2c_rdwr(write)

write = i2c_msg.write(address, [0xA1, offset])
read = i2c_msg.read(address, length)
bus.i2c_rdwr(write, read)          # Data read in list 'read'

```

3.7 Addressable LED Strip WS2812

The WS2812 (often referred to as NeoPixel in the Adafruit ecosystem) is an addressable, smart RGB LED source. Unlike traditional RGB LED strips where the entire strip changes color uniformly, every single LED on a WS2812 strip can be controlled individually and independently.

Key Technical Features:

- **Integrated Control Circuit:** Inside each 5050 (5.0mm × 5.0mm) SMD LED (Surface Mount Device Light Emitting Diode) package, there is a built-in constant-current driver IC along with high-brightness Red, Green, and Blue LED chips.
- **Single-Wire Communication:** It uses a specialized, time-critical single-line NRZ (Non-Return-to-Zero) serial protocol. Because data is passed sequentially from one pixel to the next, an entire strip of hundreds of LEDs can be driven using just one digital GPIO pin from a microcontroller or single-board computer (like a Raspberry Pi or Arduino).
- **Cascading Architecture:** Each pixel acts as a data repeater. When a pixel receives a stream of data, it strips off the first 24 bits (8 bits each for R, G, and B) for its own color configuration, reshapes the remaining data signals, and passes them along via its Data Out (DO) pin to the Data In (DI) pin of the next pixel in line.

A typical addressable LED strip implementation is illustrated in Figure 3.8. This type of LED strip exposes three physical interfacing terminals: Power (+5V), Ground (GND), and Serial Data Input (DI).

Data is clocked sequentially into the DI terminal of the primary LED pixel. The internal control logic of the first pixel intercepts the initial 24 bits of the incoming serial data stream to update its local RGB color state. It then automatically reshapes the remaining data stream and routes it via its Data Output (DO) pin to the DI terminal of the subsequent cascading pixel. Through this daisy-chained cascade architecture, an application can achieve independent color and intensity control over every individual LED on the strip.

The protocol's timing constraints are exceptionally stringent. For example, transmitting a logical bit '1' requires a high-level voltage pulse spanning 700ns followed by a low-level period of 600ns. Conversely, a logical bit '0' requires a high-level pulse spanning 350ns followed by a low-level period of 800ns. The permissible timing tolerance for these waveform windows is narrow ($\pm 150\text{ns}$).

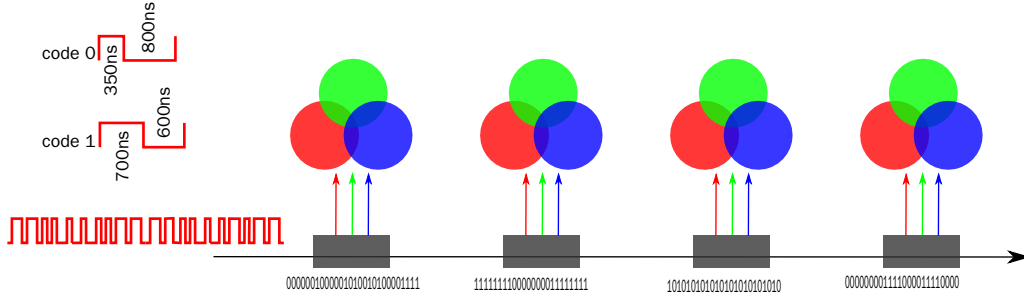


Figure 3.8: WS2812 Addressable LED Strip

Because the Linux operating system operates as a preemptive, multi-tasking time-sharing system, standard software-driven GPIO bit-banging cannot guarantee such high microsecond-level real-time deterministic accuracy. Consequently, specialized hardware blocks on the Broadcom SoC must be leveraged to synthesize the required waveforms. In practice, this is typically achieved by utilizing either the hardware **Serial Peripheral Interface** (SPI) bus serializer or the PWM controller backed by direct memory access (DMA) transfers.

To control the LED strip, we need to install `rpi_ws281x` module:

```
# git clone https://github.com/yfang1644/rpi_ws281x
# cd rpi_ws281x/python
# python3 setup.py build
# python3 setup.py install
```

This module provides both C and Python libraries. It supports the use of PWM, PCM, and SPI; however, only PWM allows for the simultaneous and independent control of two LED strips. The pin assignments are listed in Table 3.2. GPIO pins with numbers higher than 27 are not exposed on the Raspberry Pi board. Example code can be found in the ‘`python/examples/`’ directory.

Table 3.2: Supported GPIO Pins on Raspberrypi

Function	GPIO
PWM 0	12, 18, 40, 52
PWM 1	13, 19, 41, 53, 45
PCM	21, 31
SPI	10, 38

```
1 from rpi_ws281x import PixelStrip
```

```
2
3 # LED strip configuration:
4 LED_COUNT      = 16      # Number of LEDs in the strip
5 LED_PIN        = 18      # GPIO pin
6 LED_FREQ_HZ    = 800000  # Digital Frequency(800kHz)
7 LED_DMA        = 10      # DMA channel
8 LED_BRIGHTNESS = 255     # Brightness
9 LED_INVERT     = False   # Polar invert
10 LED_CHANNEL    = 0       # PWM Channel 1
11 LED_STRIP      = ws.WS2812_STRIP
12
13 # create NeoPixel object
14 strip = PixelStrip(LED_COUNT,
15                   LED_PIN,
16                   LED_FREQ_HZ,
17                   LED_DMA,
18                   LED_INVERT,
19                   LED_BRIGHTNESS,
20                   LED_CHANNEL
21                   )
22
23 strip.begin()      # Module initialize
24
25 # red, green and blue each 8 bits
26 red, green, blue = 200, 100, 150
27 color = (red << 16)|(green << 8)|blue
28
29 # Set colors for the first 4 LEDs
30 for x in range(4):
31     strip.setPixelColor(x, color)
32
33 strip.show()
```